

FINAL YEAR PROJECT REPORT

Project title:

**Design and Development of a Real-Time Queue Monitoring and
Wait-Time Estimation System**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

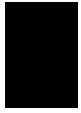
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	vii
List of Tables	ix
List of Abbreviations and Symbols	x
General Introduction	1
1 Context of the Project	2
1.1 Introduction	3
1.2 Host Organization	3
1.2.1 DataDoIt	3
1.2.2 Sectors of Activity	4
1.3 Presentation of the Project	5
1.3.1 Project Context	5
1.3.2 Problem Statement	5
1.3.3 Study of Existing Approaches	6
1.3.3.1 Queue Management Platforms	6
1.3.3.2 Sensor-Based Analytics	7
1.3.3.3 Vision-Based Monitoring Frameworks	7
1.3.4 Proposed Solution and Project Objectives	7
1.3.5 Project Scope, Boundaries, and Stakeholders	8
1.4 Project Management Methodology	9
1.4.1 Choice of Methodology: Agile	9
1.4.2 SCRUM	9

1.5	Conclusion	10
2	Requirements Specification and Analysis	11
2.1	Introduction	12
2.2	Requirements Specification	12
2.2.1	Identification of Actors	12
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	15
2.3	Product Backlog and Sprint Organization	15
2.3.1	Product Backlog	16
2.3.2	Sprint Planning	18
2.4	Global Use Case Diagram	18
2.5	Project Environment	19
2.5.1	Hardware and Software Environment	19
2.5.2	Development Tools	20
2.5.3	Technologies Used	21
2.6	Software Architecture	23
2.6.1	Overall Architecture	23
2.6.2	Backend Architecture	24
2.6.3	Frontend Architecture	25
2.7	Conclusion	26
3	Sprint 1: Perception and Queue Analytics	28
3.1	Introduction	29
3.2	Sprint Backlog	29
3.3	Design	30
3.4	Implementation	34
3.4.1	Perception Pipeline	34

3.4.2	Queue Event Windows and Rate Estimation	34
3.4.3	Wait-Time Logic and Adaptive Estimation	35
3.4.4	Runtime Metric Contract	35
3.5	Conclusion	36
4	Sprint 2: Backend Services and API Integration	37
4.1	Introduction	38
4.2	Sprint Backlog	38
4.3	Design	39
4.4	Implementation	40
4.4.1	Source Integration and Camera Discovery	40
4.4.2	API and Live Transport	41
4.4.3	Persistence, State Recovery, and System Resilience	44
4.5	Conclusion	44
5	Sprint 3: Web Dashboard and Live Monitoring	45
5.1	Introduction	46
5.2	Sprint Backlog	46
5.3	Design	48
5.3.1	Visual Design and Interface Architecture	48
5.3.2	Dashboard Data Flow and Integration Architecture	48
5.3.3	Design Rationale	50
5.4	Implementation	50
5.4.1	Project Setup and Layout Skeleton	50
5.4.2	Styling and Reusable Interface Rules	50
5.4.3	API Integration	50
5.4.4	Authentication and Role-Based Navigation	51
5.4.5	Dashboard State Orchestration and Live Update Reliability	53

5.4.6	Live Video Delivery and Transport Selection	53
5.4.7	Dashboard UI Integration	54
5.4.8	Manager Onboarding Workflow	54
5.4.9	Analytics and Historical Reporting	59
5.4.10	Integration Testing	60
5.5	Conclusion	61
6	Sprint 4: Automation, Evaluation, and Optimization	62
6.1	Introduction	63
6.2	Sprint Backlog	63
6.3	Design	64
6.4	Implementation	65
6.4.1	Alert Payload and Delivery Architecture	65
6.4.2	Telegram Integration and Notification Delivery	66
6.4.3	Asynchronous Alert Dispatch and Deduplication	67
6.4.4	n8n Workflow Structure and Validation	67
6.5	Tests and Evaluation	67
6.5.1	Evaluation Strategy and Scope	67
6.5.2	Integrated System Verification	69
6.5.3	Runtime Optimization and Performance Tuning	69
6.6	Conclusion	70
7	General Conclusion	71
7.1	Summary	72
7.2	Objective Review	73
7.3	Future Improvements	73



List of Figures

1.1	DataDoIt company logo	4
1.2	Scrum process used to organize the project.	9
2.1	Global use case diagram of the queue wait-time estimation system. . . .	19
2.2	Overall system architecture from video acquisition to alert delivery. . . .	23
2.3	MVVM architectural pattern showing the relationship between Model, View, and ViewModel components.	26
3.1	Sprint 1 activity flow from frame ingestion to queue metrics output. . . .	32
3.2	Sprint 1 analytics activity diagram for queue metric generation.	33
4.1	Sprint 2 backend class diagram showing core entities for feed orchestra- tion and persistence.	40
4.2	Camera discovery activity diagram: ONVIF discovery, RTSP resolution, and stream validation.	42
4.3	ONVIF protocol sequence: WS-Discovery broadcast, direct media service resolution, and device service indirection.	43
5.1	Dashboard data flow sequence: authentication, live metrics, fallback polling, and video transport.	49
5.2	Manager/admin sign-in screen for dashboard access.	51
5.3	Manager account creation screen.	52
5.4	Administrator manager-administration interface.	52
5.5	Manager dashboard: live surface, feed summaries, and recent activity. .	54

5.6	Manager onboarding activity diagram: feed setup, zone definition, model selection, and launch confirmation.	55
5.7	Feed setup screen used to prepare an uploaded video or ONVIF camera source before zone tracing and model selection.	56
5.8	Zone tracing screen used to define the queue polygon for the selected feed.	57
5.9	Model selection screen used to choose the YOLO size before launching the feed.	58
5.10	Review and launch panel used to confirm staged feeds before batch creation or launch.	59
5.11	Analytics dashboard: KPI summaries, trends, zone rankings, and export.	60
6.1	Sprint 4 n8n workflow for routing queue alerts through cooldown checks, archival, and Telegram delivery.	65
6.2	Example Telegram alert delivered.	66
6.3	Sprint 4 evaluation and resilience activity diagram for tests, tuning, and recovery verification.	68



List of Tables

1.1	From operational pain points to system-level requirements	6
1.2	Objectives and observable outputs	8
2.1	Functional requirement traceability matrix	14
2.2	Non-functional requirement quality matrix	15
2.3	Product backlog of the project	16
2.4	Sprint planning based on the product backlog	18
2.5	Primary global use cases	20
2.6	Hardware environment	20
2.7	Development tools used in the project	21
3.1	Sprint 1 backlog	30
4.1	Sprint 2 backlog	39
5.1	Sprint 3 backlog	47
6.1	Sprint 4 backlog	64
6.2	Runtime configuration variants and measured performance characteristics.	69



List of Abbreviations and Symbols

The following symbols and abbreviations are used throughout the report.

Symbols

λ	Arrival rate.
μ	Service rate.
W	Expected wait time.
L	Number of people currently in the queue zone.

Abbreviations

API

Application Programming Interface.

BFF

Backend for Frontend.

CSV

Comma-Separated Values.

HTTP

Hypertext Transfer Protocol.

MJPEG

Motion JPEG.

ONVIF

Open Network Video Interface Forum.

REST

Representational State Transfer.

RTSP

Real-Time Streaming Protocol.

TCP

Transmission Control Protocol.

UDP

User Datagram Protocol.

UI

User Interface.

WebRTC

Web Real-Time Communication.

WHEP

WebRTC HTTP Egress Protocol.

WS-Discovery

Web Services Discovery.



General Introduction

In many service environments, waiting time is one of the first signs that the organization is under pressure. Supermarkets, banks, and service centers may have enough staff during normal periods, but a sudden increase in demand can quickly create queues that are difficult to evaluate by observation alone.

In practice, customers usually join a queue without knowing how long they will wait. Managers and operational staff also have limited real-time information for deciding when to open another service point, redirect customers, or escalate the situation. Manual observation remains subjective, and simple counting tools do not explain how fast the queue is moving or whether the waiting time is becoming unstable.

This end-of-studies project addresses that gap through a real-time queue monitoring system based on computer vision and queueing theory. The system detects and tracks people in video streams, restricts the analysis to configurable queue zones, estimates arrival and service rates, computes waiting time, and presents the resulting indicators in a form that can support operational decisions.

It combines classical queueing theory with computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates.

The report is organized as follows. Chapter 1 presents the project context, the host organization, the problem statement, and the objectives. Chapter 2 defines the requirements and the main architecture choices. Chapters 3 to 6 describe the work sprint by sprint, from perception and queue analytics to the web dashboard and alert automation. The general conclusion closes the report and presents future work directions.

Context of the Project

Table of Content

1.1	Introduction	3
1.2	Host Organization	3
1.2.1	DataDoIt	3
1.2.2	Sectors of Activity	4
1.3	Presentation of the Project	5
1.3.1	Project Context	5
1.3.2	Problem Statement	5
1.3.3	Study of Existing Approaches	6
1.3.4	Proposed Solution and Project Objectives	7
1.3.5	Project Scope, Boundaries, and Stakeholders	8
1.4	Project Management Methodology	9
1.4.1	Choice of Methodology: Agile	9
1.4.2	SCRUM	9
1.5	Conclusion	10

1.1 Introduction

Queue monitoring is useful only when it connects the observed situation to a decision. For this reason, the project was not limited to counting people in a scene; it had to connect visual observation, waiting-time estimation, dashboard supervision, and alerting.

This chapter presents the setting in which the project was carried out: the collaboration with DataDoIt, the operational problem, the comparison with existing approaches, the solution scope, and the Scrum organization used during development.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt works on projects that connect data science and sustainability for governments, municipalities, and enterprises across the EMEA region. Its services cover decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. The company uses the motto Measure It, Optimize It, Do It to describe a working method based on analysis, planning, and deployment. At the time of the project, it had delivered more than 15 active projects across 6 sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration gave the project a concrete setting: collect useful data from live environments, transform it into indicators, and make those indicators available quickly enough for action. These concerns shaped the design and scope of the system. The DataDoIt company logo is shown in Figure 1.1.

The collaboration perimeter centered on queue management in service scenarios,



Figure 1.1: DataDoIt company logo

multi-source video ingestion, low-latency monitoring, actionable queue indicators, and iterative validation across the main system layers.

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,
- financial services firms navigating ESG compliance and green finance opportunities,
- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is subjective, hard to scale, and dependent on continuous attention. In many settings, teams react only after congestion is already visible, because they do not have a measured signal that warns them earlier.

This project addresses that gap by turning camera streams into decision-oriented telemetry. The target output includes people in zone, arrival rate, service rate, adaptive wait-time estimates, and alert states for monitoring and automation layers.

1.3.2 Problem Statement

Current operational practices and basic counting tools still have recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they provide weak visibility into queue stability during sudden demand spikes,
- they are often disconnected from automated response workflows.

As a result, interventions such as staff reallocation, lane opening, or escalation can be delayed or triggered from incomplete information. The problem is therefore both analytical and practical: the system must estimate queue behavior and deliver the result to the people who can act on it. Table 1.1 maps these pain points to system-level requirements.

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between staff members, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
Queue behavior under load spikes is hard to interpret.	Teams may overreact or underreact when service and arrivals fluctuate quickly.	Adaptive wait-time formula that responds gracefully to changing queue conditions.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

Table 1.1: From operational pain points to system-level requirements

1.3.3 Study of Existing Approaches

Several computer-vision frameworks provide the basic building blocks for person detection and tracking. In parallel, queueing theory gives a formal way to describe service-system behavior and relate arrival rate, service rate, and expected waiting time [1].

In practice, existing commercial and open-source tools cover only parts of the queue-management problem. They fall into three broad families: queue management platforms, sensor-based analytics, and vision-based monitoring frameworks.

1.3.3.1 Queue Management Platforms

Platforms such as Waitwhile [3] and Qminder [4] focus on active queue management. They let customers join virtual queues through kiosks, QR codes, or web links, and they notify staff through SMS or dashboards. Their strength is customer-facing experience: reduced perceived waiting, appointment scheduling, and service routing. However,

they require explicit customer interaction (check-in) and do not observe the physical queue directly. They cannot estimate wait time from the scene itself, and they do not detect congestion unless a customer has already registered.

1.3.3.2 Sensor-Based Analytics

RetailNext [5] and Xovis [6] represent the sensor-based analytics family. RetailNext uses ceiling-mounted AI sensors to track foot traffic, measure conversion rates, and optimize staffing in retail environments. Xovis uses 3D optical sensors for precise people flow and queue length measurement, particularly in airports and large retail spaces. Both provide high-accuracy occupancy and dwell-time data, but they require proprietary hardware installations and are primarily designed for large-scale enterprise deployments. They observe the queue passively but do not typically integrate with external automation workflows or provide open APIs for custom alert routing.

1.3.3.3 Vision-Based Monitoring Frameworks

Open-source tools built on YOLO detection and tracking, such as those provided by Ultralytics [2] and Roboflow [7], offer the computer-vision building blocks needed for queue monitoring: person detection, tracking, and zone-based counting. These frameworks are flexible and can be adapted to specific use cases. However, they typically stop at detection and counting. They do not include queue-theoretic estimation of wait time, stability classification, or integration with notification workflows. The gap between raw detection output and actionable operational metrics is left to the developer.

1.3.4 Proposed Solution and Project Objectives

The proposed system links perception, analytics, monitoring, and automation in a modular architecture. Instead of stopping at raw visual observations, it maps camera feeds to queue-state information, then makes that information available through dashboard

views and alert workflows.

To establish clear traceability between the technical solution and measurable project outcomes, the objectives and their observable outputs are listed in Table 1.2.

Objective ID	Objective	Observable output
OBJ-01	Real-time detection and tracking	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Adaptive wait-time estimation	Compute wait time with adaptive logic that responds to queue load.
OBJ-05	Web monitoring	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Webhook-based alert routing through n8n and Telegram.

Table 1.2: Objectives and observable outputs

1.3.5 Project Scope, Boundaries, and Stakeholders

The implementation scope covers three distinct but interdependent domains: perception and queue analytics, runtime services and integration, and web-based operational interfaces. The report follows the same structure so that design decisions and validation activities remain easy to trace.

The system is designed for three stakeholder groups with complementary responsibilities. Administrators manage account governance and access controls. Managers configure feeds and thresholds, monitor live queue status, respond to alerts, and review performance indicators. End users benefit indirectly through shorter waiting times and smoother service flow. This separation keeps each stakeholder focused on the part of the system they actually use.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected because the project combined several domains that had to work together: computer vision, queue analytics, runtime services, frontend workflows, and automation. Building all of them in a single delivery cycle would have made integration risks harder to detect.

Iterative delivery made it possible to validate the project progressively: first the perception output, then the queue metrics, then the live dashboard behavior, and finally the alert workflow and system hardening.

1.4.2 SCRUM

Development followed a sprint-based Scrum methodology with validation at the end of each major increment. Each sprint focused on a coherent part of the product: perception and analytics first, then camera connectivity and live services, then the web dashboard, and finally alert automation and evaluation. For readability and traceability, the work is presented through three logical modules, as illustrated in Figure 1.2.

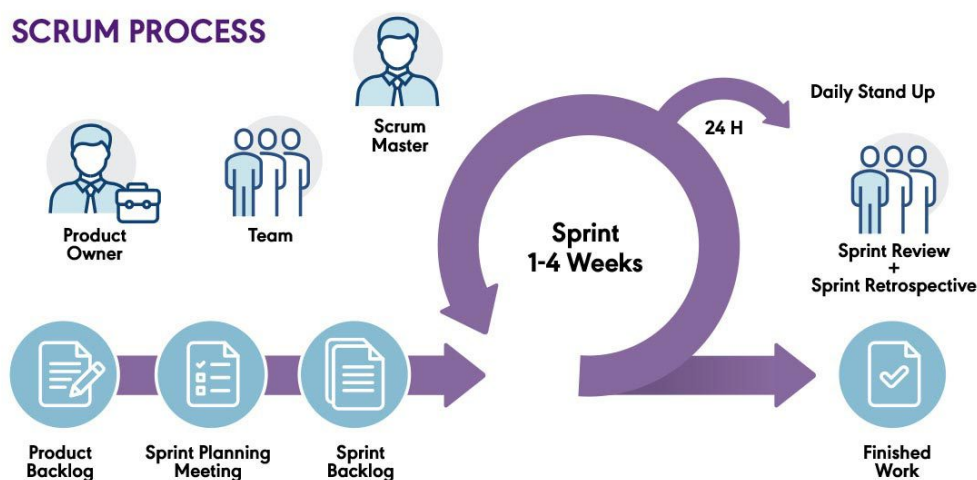


Figure 1.2: Scrum process used to organize the project.

1.5 Conclusion

This chapter defined the project context and the delivery strategy. Chapter 2 builds on this baseline by presenting the requirements, architecture constraints, and analysis artifacts used to guide the implementation.

Table of Content

2.1	Introduction	12
2.2	Requirements Specification	12
2.2.1	Identification of Actors	12
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	15
2.3	Product Backlog and Sprint Organization	15
2.3.1	Product Backlog	16
2.3.2	Sprint Planning	18
2.4	Global Use Case Diagram	18
2.5	Project Environment	19
2.5.1	Hardware and Software Environment	19
2.5.2	Development Tools	20
2.5.3	Technologies Used	21
2.6	Software Architecture	23
2.6.1	Overall Architecture	23
2.6.2	Backend Architecture	24
2.6.3	Frontend Architecture	25
2.7	Conclusion	26

2.1 Introduction

After defining the project context, the next step was to translate the idea into requirements that could guide the work. This chapter identifies the actors, lists the expected functions and quality constraints, and shows how those requirements are distributed across the implementation plan.

The requirements follow the same queue-monitoring path used throughout the project: video ingestion, person detection and tracking, queue-metric estimation, dashboard delivery, authentication, and alert automation.

2.2 Requirements Specification

2.2.1 Identification of Actors

The platform involves two actors inside the application and one operational actor outside it. This distinction matters because the software can detect, estimate, and notify, but the physical response to congestion still happens on site.

The actors are:

- **Administrator:** manages manager accounts (create, update, delete) and controls access governance.
- **Manager:** configures sources and thresholds, monitors live queues, reacts to alerts, and reviews performance indicators.
- **On-site Operator (external actor):** receives queue alert notifications in a shared Telegram group and applies field actions, such as opening a new queue line or redirecting customer flow.

This split keeps the functional scope clear: access control, monitoring, and alert notification remain inside the system, while physical queue intervention and manual

completion confirmation are treated as external operational actions.

2.2.2 Functional Requirements

At a high level, the system captures a feed, counts people inside a queue zone, estimates queue behavior, updates the dashboard, and sends alerts when the configured conditions are reached. These requirements define the minimum end-to-end behavior expected from the platform and are formalized in Table 2.1.

REQUIREMENTS SPECIFICATION AND ANALYSIS

ID	Requirement	Priority	Verification
FR-01	Detect people in live or recorded streams in near real time.	High	Validate detections and overlays in the perception layer.
FR-02	Preserve identity continuity across frames for queue analysis.	High	Validate tracker ID continuity in runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate polygon inclusion in the zone-management layer.
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in analytics.
FR-05	Compute wait time with adaptive logic that switches between precise and conservative estimates.	High	Validate formula branches and fallback behavior.
FR-06	Expose feed control and live updates through API and WebSocket channels.	High	Validate endpoint and runtime behavior in API tests.
FR-07	Dispatch alert payloads to automation workflows.	Medium	Validate webhook dispatch and n8n branching.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate persistence behavior in runtime and database checks.
FR-09	Allow manager registration, sign-in, refresh, and logout through secure cookies.	High	Validate register, login, refresh, logout, and session checks.
FR-10	Allow administrator to create, update, deactivate, and reset manager accounts.	High	Validate account-governance endpoints and audit events.
FR-11	Enforce password policy, logout after repeated failures, and role-based protection on operational endpoints.	High	Validate failed-login counters, logout behavior, and unauthorized access.
FR-12	Record authentication and account-governance actions for traceability.	Medium	Validate audit-log entries during register and login flows.
FR-13	Protect administrative and operational APIs with role-based access control.	High	Validate unauthorized access paths and audit events.

Table 2.1: Functional requirement traceability matrix

2.2.3 Non-functional Requirements

In addition to functional behavior, the platform must satisfy quality constraints that keep it usable in a live monitoring environment. A correct wait-time formula is not enough if the video feed freezes, the dashboard stops updating, or alerts are lost. For this reason, performance, reliability, and security are treated as project requirements and summarized in Table 2.2.

ID	Quality	Target indicator	Validation
NFR-01	Performance	Keep the loop near real time and limit alert delay.	Runtime observation under representative feed scenarios.
NFR-02	Reliability	Recover gracefully from camera drops and webhook failures.	Reconnect paths and retry/fallback test scenarios.
NFR-03	Security	Restrict integration access through secrets and credentials.	Webhook secret validation and credential checks.

Table 2.2: Non-functional requirement quality matrix

2.3 Product Backlog and Sprint Organization

The product backlog was prioritized according to business value and technical dependency, then organized into four sprints. The first sprint focused on perception and queue analytics because the dashboard and alerts depend on stable metrics. The second sprint added source management and live services. The third sprint turned those services into a usable web dashboard. The final sprint connected alerts to n8n and Telegram, then evaluated the complete system.

2.3.1 Product Backlog

The product backlog groups the global functionalities of the application into user stories.

These stories are prioritized with the MoSCoW method:

- **M – Must have:** essential requirements without which the project would not meet its objective.
- **S – Should have:** important requirements that improve the solution but are not blocking for the core system.
- **C – Could have:** desirable enhancements that add comfort or completeness when time permits.
- **W – Would have:** requirements deliberately left outside the current scope and reserved for future improvement.

Table 2.3 presents the product backlog used to organize the project.

Table 2.3: Product backlog of the project

ID	Epic	User Story	Priority
PB-01	E1: Perception and queue analytics	As a manager, I want people detected in live or recorded video so that queue observations start from a reliable visual signal.	M
PB-02	E1: Perception and queue analytics	As a manager, I want the same person tracked across frames so that queue counts remain consistent.	M
PB-03	E1: Perception and queue analytics	As a manager, I want counting restricted to polygon queue zones so that irrelevant movement does not affect the queue state.	M
PB-04	E1: Perception and queue analytics	As a manager, I want arrival rate, service rate, and wait time computed from tracked events so that the dashboard can interpret queue intensity.	M
PB-05	E1: Perception and queue analytics	As a manager, I want adaptive wait-time estimation so that the output remains bounded and useful when the queue becomes unstable.	M

REQUIREMENTS SPECIFICATION AND ANALYSIS

ID	Epic	User Story	Priority
PB-06	E2: Backend services and live integration	As a manager, I want IP cameras discovered and connected securely so that live sources can be onboarded without hardcoded stream addresses.	M
PB-07	E2: Backend services and live integration	As a manager, I want feed lifecycle actions exposed through the API so that feeds can be started, stopped, updated, and recovered centrally.	M
PB-08	E2: Backend services and live integration	As a manager, I want live metrics and feed status delivered through real-time channels so that monitoring reflects the current situation.	M
PB-09	E2: Backend services and live integration	As a manager, I want fallback video delivery when the preferred transport is unavailable so that monitoring remains usable.	S
PB-10	E3: Web dashboard and access control	As a manager, I want a coherent dashboard interface so that setup, monitoring, and analytics can be used from one place.	M
PB-11	E3: Web dashboard and access control	As an administrator, I want to create and maintain manager accounts so that access is governed centrally.	M
PB-12	E3: Web dashboard and access control	As a manager, I want protected sign-in and role-based navigation so that each user sees only the screens matching their responsibility.	M
PB-13	E3: Web dashboard and access control	As a manager, I want live dashboard updates and fallback polling so that the interface remains informative during temporary interruptions.	S
PB-14	E3: Web dashboard and access control	As a manager, I want historical analytics and exportable indicators so that repeated congestion patterns can be reviewed.	C
PB-15	E4: Alert automation and evaluation	As a manager, I want alert payloads dispatched to n8n so that operational staff can be notified automatically.	M
PB-16	E4: Alert automation and evaluation	As a manager, I want Telegram notifications routed to the on-site operator so that field action can be triggered through a familiar channel.	M

ID	Epic	User Story	Priority
PB-17	E4: Alert automation and evaluation	Add webhook retries, cooldown handling, and delivery logging so that notification failures are visible and repeated alerts are controlled.	S
PB-18	E4: Alert automation and evaluation	Run integrated tests and apply runtime tuning so that the finished system is more reliable under realistic conditions.	S
PB-19	E5: Future improvement	As a manager, I want multi-zone and multi-server queue modeling so that parallel cashier configurations can be represented more precisely.	W

2.3.2 Sprint Planning

Based on the product backlog, the project was divided into four sprints. Table 2.4 summarizes the sprint plan and the epic covered by each sprint.

Sprint ID	Sprint	Epic ID	Duration
Sprint 1	Perception and Queue Analytics	E1	4 weeks
Sprint 2	Backend Services and API Integration	E2	4 weeks
Sprint 3	Web Dashboard and Live Monitoring	E3	4 weeks
Sprint 4	Automation, Evaluation, and Optimization	E4	4 weeks

Table 2.4: Sprint planning based on the product backlog

2.4 Global Use Case Diagram

The global use case diagram gives a high-level view of how the administrator and manager interact with the platform during configuration, monitoring, and alert handling. It is shown in Figure 2.1.

Use Case Diagram: Chapter 2

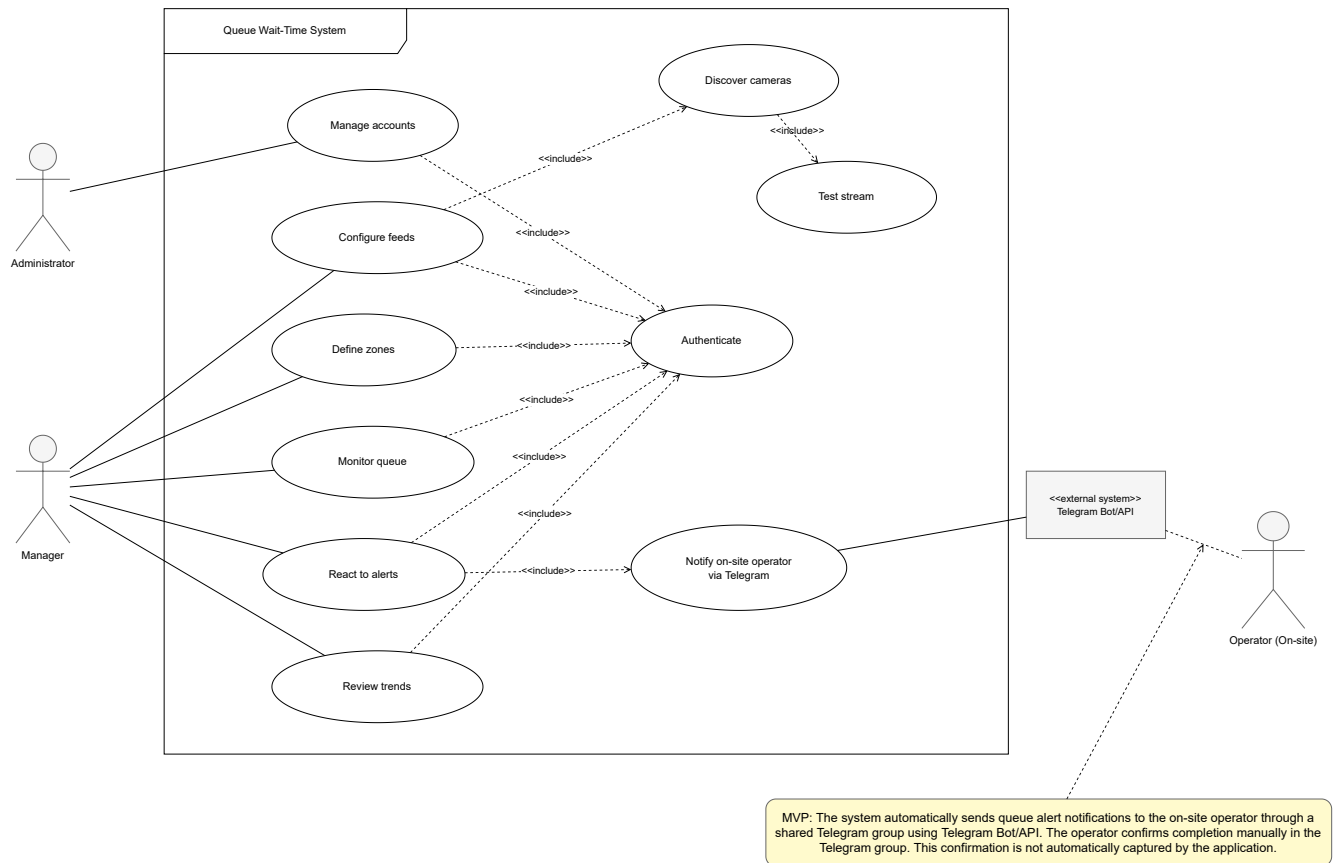


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At the system level, the use cases can be grouped into four interactions: configuration, live monitoring, analytics interpretation, and alert handling/resolution, with the main actor responsible for each use case listed in Table 2.5.

2.5 Project Environment

2.5.1 Hardware and Software Environment

The hardware environment used during development is shown in Table 2.6.

Use case ID	Use case summary	Primary actor
UC-01	Create, update, or remove manager accounts.	Administrator
UC-02	Register and configure a video feed source.	Manager
UC-03	Define or update queue zone geometry.	Manager
UC-04	Monitor live queue metrics and feed status.	Manager
UC-05	Receive and interpret warning or critical alerts.	Manager
UC-06	Automatically notify the on-site operator via Telegram when an alert is detected.	System automatic behavior
UC-07	Review trends and performance indicators.	Manager

Table 2.5: Primary global use cases

Component	Type
Processor	Intel Core i5 12th Generation
Memory (RAM)	16 GB
Graphics card	NVIDIA GeForce RTX 4060
Storage	512 GB SSD

Table 2.6: Hardware environment

2.5.2 Development Tools

The project also relied on a small set of tools for editing, versioning, diagramming, container support, and report generation. Keeping these tools separate from the technology stack makes it easier to distinguish the working environment from the components used by the final system, as listed in Table 2.7.

Tool	Role in the project	Typical use
VS Code [8]	Main editor for the backend, frontend, and report sources.	Day-to-day implementation, review, and navigation.
Git / GitHub [9], [10]	Version control and change history tracking.	Commit management, branching, and collaboration.
draw.io [11]	Diagram authoring for architecture and UML models.	Use case, UML class, and system architecture diagrams.
Docker / Docker Compose [12]	Local service orchestration and reproducible setup.	Supporting services such as MediaMTX and n8n.

Table 2.7: Development tools used in the project

2.5.3 Technologies Used

The technology stack follows the main stages of the system, from video perception through to automation and persistence. Each technology is described below.

YOLO26 [2] is a real-time object detection model that processes entire images in a single forward pass through a convolutional neural network. In this project, it serves as the perception backbone, detecting people in video frames with high speed and accuracy.

ByteTrack [13] is a multi-object tracking algorithm that associates detections across frames using both high- and low-confidence bounding boxes, reducing identity switches. It is used here to maintain persistent person identities throughout the video stream, which is essential for accurate queue measurement.

OpenCV [14] is an open-source computer vision library providing tools for image processing, video capture, and graphical annotation. It handles frame acquisition from cameras and uploaded videos, and renders visual overlays such as bounding boxes and zone boundaries.

FastAPI [15] is a modern Python web framework designed for building high-performance HTTP and WebSocket APIs with automatic data validation and interactive

documentation. It exposes the system's endpoints and coordinates communication between the vision pipeline, analytics engine, and frontend.

Pydantic v2 [16] is a data validation library that enforces strict typing on request and response models at runtime. It is used alongside FastAPI to ensure that all data exchanged between system components conforms to defined schemas.

React [17] is a JavaScript library for building component-based user interfaces. Combined with TypeScript [18] for static typing, Vite [19] as the build toolchain, and Tailwind CSS [20] for utility-based styling, it forms the foundation of the monitoring dashboard.

WebRTC [21] is a browser standard enabling low-latency audio and video streaming directly in the browser without plugins. It is used to deliver live camera feeds to the dashboard, while MediaMTX [22] acts as the WebRTC gateway bridging the camera source to the browser, with MJPEG serving as a simpler fallback for uploaded video files.

n8n [23] is an open-source workflow automation platform that connects services through configurable pipelines. It routes queue alerts generated by the backend and delivers them as operational messages via the Telegram Bot API [24].

SQLite [25] is a lightweight embedded relational database that requires no separate server process. It stores queue metrics, session metadata, and alert history locally alongside the application for later analysis.

NumPy, **Pandas**, and **SciPy** [26], [27], [28] are foundational Python scientific computing libraries used in the analytics pipeline. NumPy supports numerical array operations, Pandas handles structured metric storage and manipulation, and SciPy provides statistical routines. Wait-time estimation is derived from an M/M/1 queuing model [1], which approximates queue behavior under Poisson arrivals and exponential service times.

This selection supports the complete path from perception through automation while keeping each stage understandable on its own.

2.6 Software Architecture

2.6.1 Overall Architecture

The architecture is organized into four stages that match the flow of data through the system, from input acquisition to output delivery. Figure 2.2 illustrates this flow.

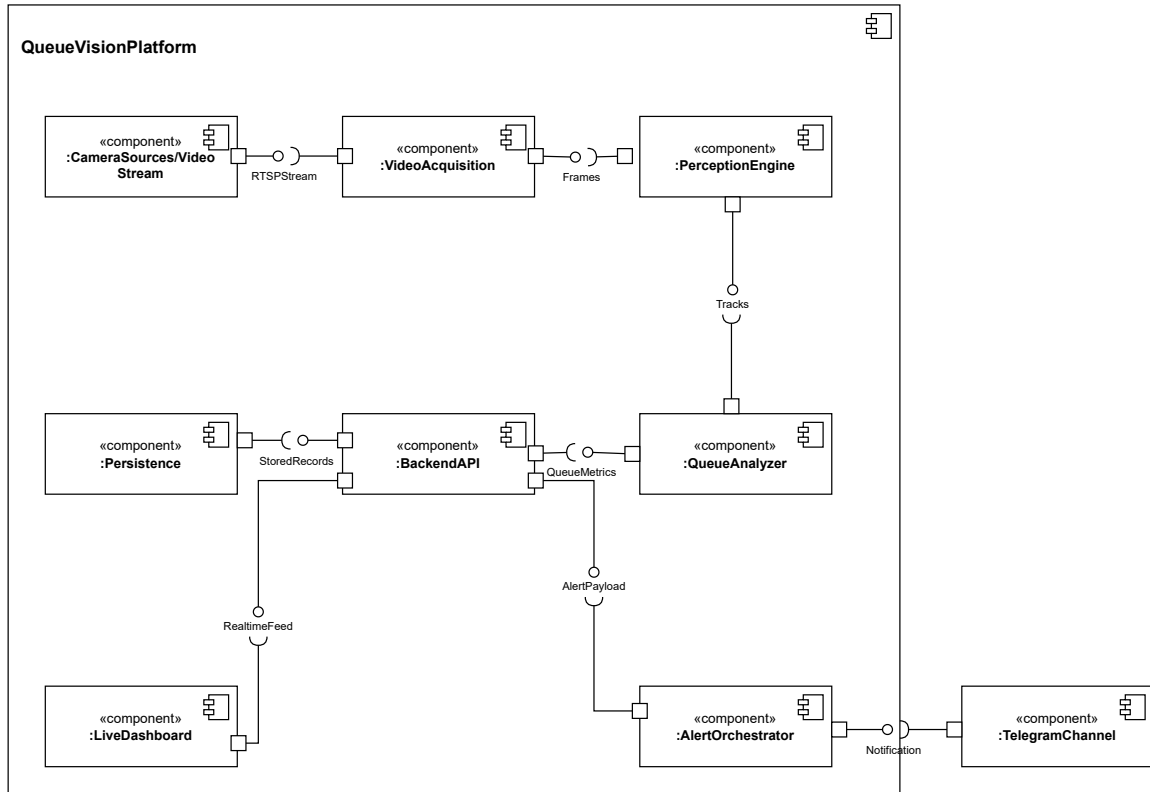


Figure 2.2: Overall system architecture from video acquisition to alert delivery.

1. **Input acquisition:** multiple sources feed into the acquisition component, which handles connection management and stream ingestion.
2. **Processing:** the processing engine applies algorithms to extract structured observations from raw input data and exposes them through a defined interface.
3. **Analysis:** the analysis module receives structured observations and computes quantitative metrics using domain-specific models and evaluation logic.
4. **Output and integration:** the backend acts as the distribution hub that routes computed metrics across three outputs: the live interface via real-time channels,

the persistence layer for historical records, and the orchestrator which forwards events to external notification services.

This separation keeps the four stages easier to validate. Analysis formulas depend on processing outputs, the backend depends on the metric contract, and the interface, persistence, and orchestration consume the same outputs without being tightly coupled to one another.

2.6.2 Backend Architecture

The backend adopts a three-tier layered architecture combined with a Backend-for-Frontend (BFF) pattern. The layered architecture organizes the system vertically into separate tiers based on responsibility, while the BFF pattern tailors the backend API specifically to the requirements of the frontend client, rather than serving as a generic intermediary.

It is composed of three main layers:

- **Presentation layer:** serves as the entry point, exposing HTTP endpoints and real-time communication channels that receive client requests, validate input data, and return structured responses suitable for direct consumption by the user interface.
- **Application layer:** contains the orchestration and business logic. It coordinates operations, enforces rules, manages workflows, evaluates conditions, and broadcasts events across the system.
- **Data and integration layer:** handles persistence and external service communication. It manages data storage operations and abstracts the details of third-party service interactions through dedicated integration adapters.

The three-tier layered architecture offers several benefits such as:

- **Separation of concerns:** each layer has a single responsibility, making the system easier to understand and maintain.

- **Improved testability:** layers can be tested independently by mocking dependencies between them.
- **Flexibility:** changes in one layer have minimal impact on the others, allowing independent evolution of persistence, business logic, and API contracts.

The Backend-for-Frontend pattern further enhances this architecture by shaping the presentation layer around the specific needs of the dashboard interface. This approach groups related operations together, reduces the number of client requests, and delivers aggregated responses optimized for the frontend use cases, ultimately simplifying the client-side logic.

2.6.3 Frontend Architecture

The frontend adopts the Model-View-ViewModel (MVVM) architectural pattern, an evolution of the classical Model-View-Controller (MVC) pattern in which the Controller is replaced by a dedicated ViewModel component. The ViewModel acts as a bridge between the Model and the View, facilitating data binding and separating presentation logic from the user interface. It is composed of:

- **Model:** represents the data and business logic of the application. It defines the data structures, service contracts, and the way the application interacts with backend services and external data sources.
- **View:** represents the application's user interface. It is responsible for displaying information to the user and capturing user interactions through visual components and layouts.
- **ViewModel:** acts as the intermediary between the Model and the View. On one hand, it transforms and prepares data for presentation and updates the View through data binding mechanisms. On the other hand, it processes user interactions captured by the View and updates the Model accordingly.

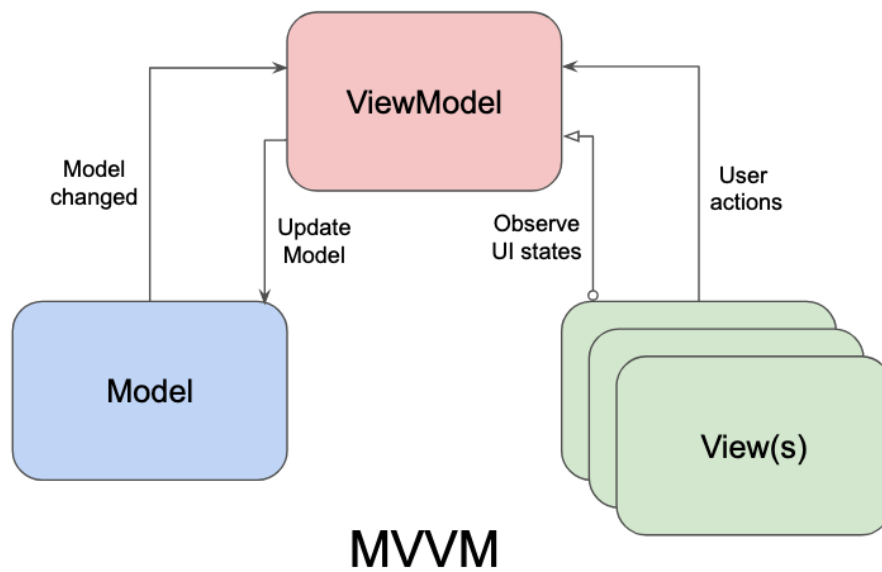


Figure 2.3: MVVM architectural pattern showing the relationship between Model, View, and ViewModel components.

The MVVM design pattern was chosen for its advantages such as:

- **Improved testability:** ViewModels can be tested independently of the View, enabling unit testing of presentation logic without UI dependencies.
- **Separation of concerns:** the pattern clearly distinguishes between data management, business logic, and user interface presentation, making the codebase easier to maintain and evolve.
- **Data binding support:** the ViewModel facilitates automatic synchronization between the View and the Model, reducing boilerplate code and improving consistency.

2.7 Conclusion

This chapter established the functional baseline, the quality constraints, and the architecture choices that guide the rest of the report. The next chapters show how these

requirements were implemented through perception, connectivity, dashboard monitoring, automation, and evaluation work.

Table of Content

3.1	Introduction	29
3.2	Sprint Backlog	29
3.3	Design	30
3.4	Implementation	34
3.4.1	Perception Pipeline	34
3.4.2	Queue Event Windows and Rate Estimation	34
3.4.3	Wait-Time Logic and Adaptive Estimation	35
3.4.4	Runtime Metric Contract	35
3.5	Conclusion	36

3.1 Introduction

The first sprint answered the most basic question of the project: can a video feed be transformed into queue indicators that are stable enough for later use? The work therefore started with people detection, frame-to-frame tracking, and zone filtering before moving to arrival rate, service rate, and wait-time estimation.

Delivering perception and analytics together was important because the later dashboard and alerting work needed more than raw counts. They needed a consistent metric output that already represented the queue zone, the tracked identities, and the estimated flow of people.

3.2 Sprint Backlog

Sprint 1 had two linked objectives. The first was to obtain a reliable perception baseline by detecting people, preserving identity across frames, and ignoring movement outside the queue zone. The second was to convert those observations into operational metrics that a manager could interpret later in the dashboard. Table 3.1 summarizes the sprint backlog by use case and links each user-facing need to its implementation work.

UC ID	User Story	Implementation Work
UC-1.1	As a manager, I want people detected in live or recorded video so that queue observations start from a reliable visual signal.	Configure detection for live and recorded video, then connect frame capture, detection, tracking, zone filtering, and display into one continuous flow.
UC-1.2	As a manager, I want the same person tracked across frames so that queue counts remain consistent.	Integrate tracking to preserve identity across consecutive frames and expose runtime settings that adapt the pipeline without code changes.
UC-1.3	As a manager, I want counting restricted to polygon queue zones so that irrelevant movement does not affect the queue state.	Define queue zones as polygons and filter detections by zone.
UC-1.4	As a developer, I want the analytics engine to compute arrival rate, service rate, and wait time from tracked events so that operational metrics are accurate and available for the dashboard.	Group tracked events into short windows and compute arrival and service rates.
UC-1.5	As a developer, I want the wait-time algorithm to adapt between M/M/1 and occupancy-based estimation so that metrics remain stable under varying queue conditions.	Apply the M/M/1 formula when arrivals are below service capacity, switch to occupancy-based estimation otherwise, and guard against empty windows, low counts, and division by zero.

Table 3.1: Sprint 1 backlog

3.3 Design

Perception and Analytics Flow

The Sprint 1 workflow can be read at two levels: the overall path from frame ingestion to metric output, and the detailed transformation from tracked identities to queue rates. This distinction helped keep the visual processing concerns separate from the waiting-time calculation.

At the perception level, the system handles three key scenarios: when a person enters the queue zone, they are treated as a possible arrival; when the same person appears in consecutive frames, the system maintains stable identification to avoid double-counting; and when people appear outside the zone, they are filtered out to keep measurements focused on the queue area.

At the queue-operation level, the system handles three practical situations. When arrivals remain below service capacity, the M/M/1 formula provides a rate-based estimate. When arrivals approach or exceed service capacity, the estimate switches to a conservative occupancy-based value. When no service events are available, the output remains bounded instead of producing unstable values. This adaptive behavior keeps a single wait-time indicator usable across changing queue conditions.

At the high level, Figure 3.1 shows the end-to-end flow from a video source (local file or live camera stream) through detection, tracking, zone filtering, and finally to queue analysis that computes arrival rate, service rate, and expected wait time.

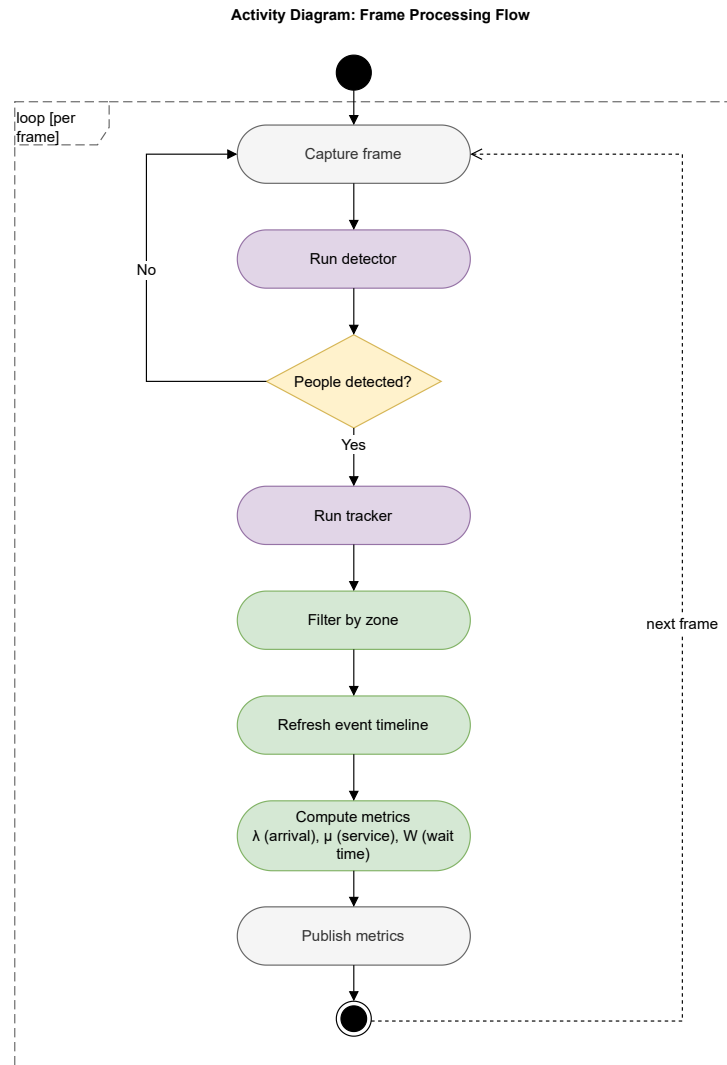


Figure 3.1: Sprint 1 activity flow from frame ingestion to queue metrics output.

At the detailed level, Figure 3.2 focuses on the queue analytics transformation that converts tracked people inside the zone into the metric payload consumed by the dashboard and later automation layers. The analytics step compares current tracked identifiers with the previous observation window to identify arrivals and departures. These events are stored in short windows, then converted into rates (λ and μ). Before applying the wait-time formula, the system checks whether enough events were observed, whether μ is positive, and whether arrivals remain below service capacity. If these guards are satisfied, it uses the M/M/1 expression; otherwise, it falls back to a bounded occupancy-based estimate.

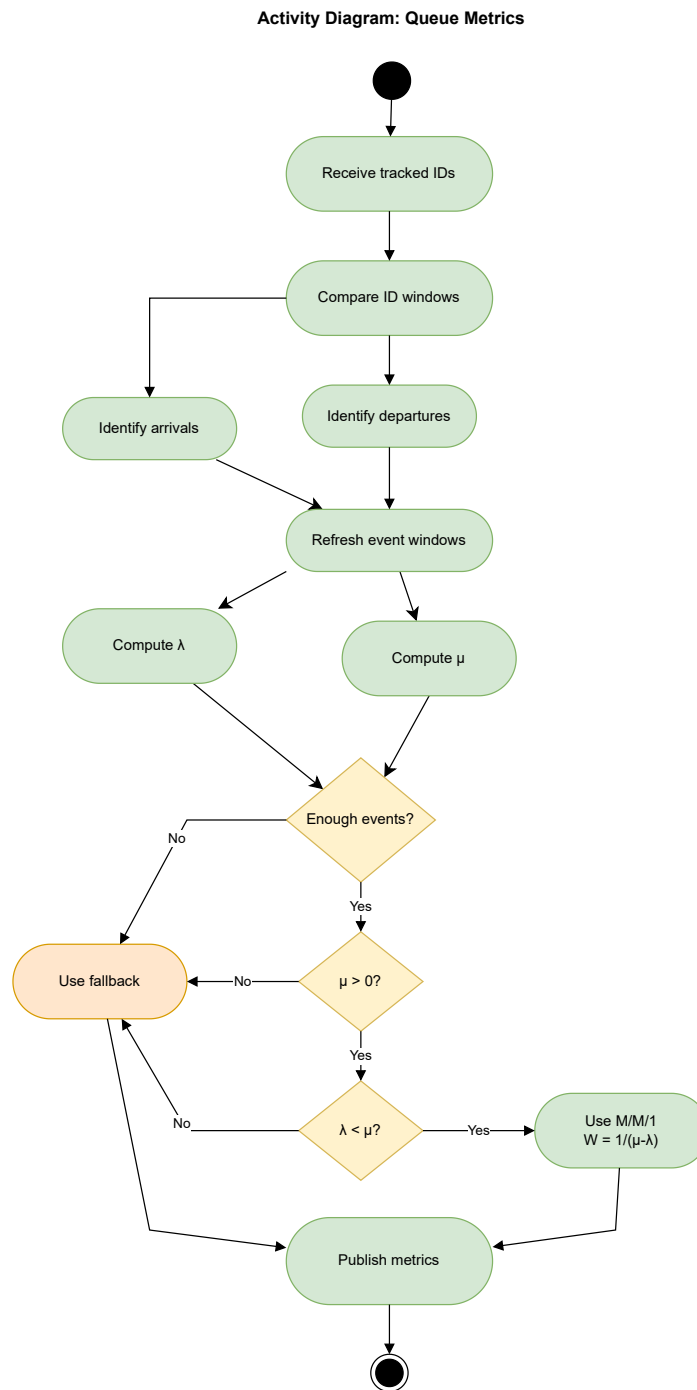


Figure 3.2: Sprint 1 analytics activity diagram for queue metric generation.

3.4 Implementation

3.4.1 Perception Pipeline

The perception pipeline combines detection, tracking, and zone filtering. YOLO identifies people in each frame, while ByteTrack preserves identity across consecutive frames. This identity continuity is necessary for arrival and departure counting, because a person should not be counted again simply because they appear in the next frame. Zone-aware filtering then restricts the measurement to the polygon-defined queue area, so unrelated movement elsewhere in the scene does not affect the queue metric.

The same processing path supports local files, webcams, and live camera streams. It can adjust frame-processing frequency, display overlays for detections, tracker identities, queue zones, and computed metrics, or run without visual output when the results are needed by later system components.

3.4.2 Queue Event Windows and Rate Estimation

The queue-analysis step uses short time windows to count arrivals and departures. Let N_a and N_d be the number of events observed in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

From these counts, the system estimates the arrival rate and the service rate. To avoid unstable values at the beginning of a run, the implementation waits until enough events have been observed before returning non-zero rates.

3.4.3 Wait-Time Logic and Adaptive Estimation

The system estimates wait time using an adaptive formula that responds to queue conditions:

$$W = \begin{cases} \frac{1}{\mu - \lambda} & \text{if } \lambda < \mu \text{ and } \mu > 0 \\ \frac{L}{\mu} & \text{otherwise} \end{cases}$$

where L is the current number of people in the zone. We use the M/M/1 sojourn-time expression because the project models each monitored lane as a single service channel with one effective queue, which matches the cashier-style setting studied in the report. Other models were less appropriate here: M/M/c would assume multiple parallel servers, which is not the default lane setup, and Little's Law is useful for validation but does not directly produce a forward wait-time estimate from the instantaneous rates alone.

The adaptive formula keeps the output interpretable. Under normal load, the estimate comes from the rate-based expression. Under overload, it falls back to a conservative value based on the current queue size. When no service events are observed, the result is bounded instead of allowing division by zero or an unrealistic wait-time value.

3.4.4 Runtime Metric Contract

The final metric output includes:

- point estimates (λ, μ, W) ,
- people-in-zone occupancy.

These values are then reused by logs, overlays, webhook payloads, and dashboard events [1], [14].

3.5 Conclusion

Sprint 1 delivered the first complete path from raw video to queue indicators. Detection, tracking, and zone filtering produced a usable perception baseline, while the queue analytics step converted tracked observations into arrival rates, service rates, and adaptive wait-time estimates.

Table of Content

4.1	Introduction	38
4.2	Sprint Backlog	38
4.3	Design	39
4.4	Implementation	40
4.4.1	Source Integration and Camera Discovery	40
4.4.2	API and Live Transport	41
4.4.3	Persistence, State Recovery, and System Resilience	44
4.5	Conclusion	44

4.1 Introduction

Sprint 2 moved the project from a local video-processing pipeline to a service that could manage several feed sources. The main challenge was no longer only computing queue metrics, but making camera onboarding, feed control, live events, and video delivery available to the rest of the system in a consistent way.

This sprint also prepared the ground for the web dashboard. Managers would later need to discover cameras, configure sources, start or stop feeds, and keep watching a stream even when the preferred transport was unavailable.

4.2 Sprint Backlog

The sprint objectives focused on four manager needs: secure connection to IP cameras, automatic camera discovery, centralized feed management, and reliable live delivery. These needs are captured in Table 4.1.

UC ID	User Story	Implementation Work
UC-2.1	As a developer, I want to implement secure IP camera stream connections with credential handling, RTSP transport control, and reconnection behavior so that live sources are reliable and reusable.	Add credential handling, RTSP transport control, reconnection behavior, and snapshot capture for camera feeds.
UC-2.2	As a manager, I want cameras discovered automatically so that stream addresses do not need to be hardcoded.	Detect ONVIF cameras, resolve media services, and retrieve stream addresses.
UC-2.3	As a developer, I want to design and implement the feed management API with lifecycle actions, metadata handling, persistence, and state recovery so that feed operations are consistent and reliable.	Expose feed control, metadata handling, diagnostics, persistence, and state recovery through typed backend endpoints.
UC-2.4	As a manager, I want live metrics and feed status delivered through real-time channels so that monitoring reflects the current situation.	Support real-time events and WebRTC delivery for IP cameras and MJPEG delivery for local videos.
UC-2.5	As a manager, I want fallback video delivery when the preferred transport is unavailable so that monitoring remains usable.	Support MJPEG delivery as a fallback when WebRTC is unavailable, and report the active transport explicitly so that the dashboard can reflect the current delivery mode.

Table 4.1: Sprint 2 backlog

4.3 Design

Backend Domain Model and Class Structure

The backend model was organized around the entities that appear in a real monitoring scenario: establishments, managers, feed configurations, video sessions, queue metrics, and alerts. Establishments group both managers and feeds, while managers operate only on the feeds assigned to their scope. This organization keeps access control

and feed recovery consistent when the system restarts.

The core entities and their relationships are illustrated in Figure 4.1.

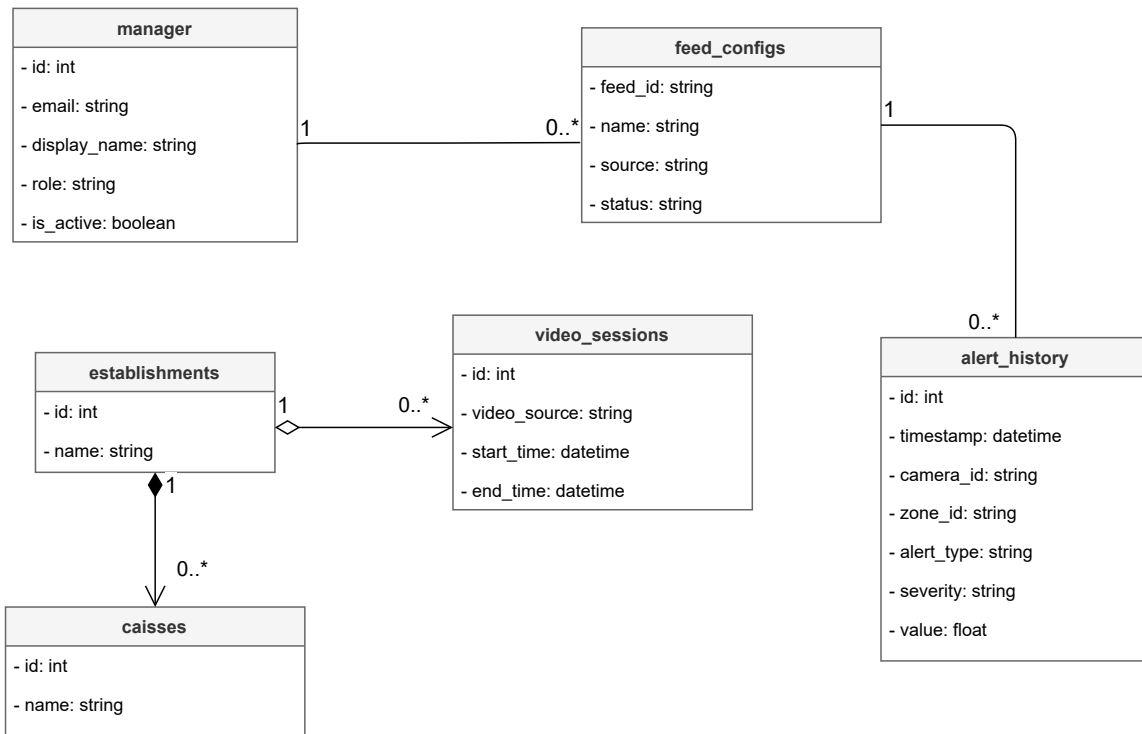


Figure 4.1: Sprint 2 backend class diagram showing core entities for feed orchestration and persistence.

4.4 Implementation

4.4.1 Source Integration and Camera Discovery

The system supports several source types, but IP cameras required the most careful handling. RTSP streams need credentials, transport selection, connection testing, and reconnection behavior. Local files and webcams were kept on simpler paths so that camera-specific constraints would not complicate every source.

Camera discovery uses ONVIF to locate devices and resolve stream information without manually entering hardcoded addresses. The workflow starts with WS-Discovery over multicast UDP, then resolves media services and RTSP stream URIs. It also handles different ONVIF camera behaviors: some cameras expose media services directly,

while others require an intermediate capability lookup before the stream address can be requested. The workflow was validated against a Digital Watchdog IP camera on a shared network, which confirmed that discovery, stream resolution, and validation could work in a realistic setup.

The discovery process gives the manager practical actions: find available cameras, test a connection, resolve the stream, and capture a snapshot before defining the queue zone. Credentials are handled without exposing them in logs, and the transport choice can adapt to the network conditions and source type.

The logical workflow of this discovery process is illustrated in Figure 4.2.

The protocol-level details are captured in Figure 4.3, which illustrates the three-phase ONVIF sequence: multicast UDP discovery, direct media service access, and device service indirection.

4.4.2 API and Live Transport

The backend exposes REST and WebSocket interfaces for feed management, source onboarding, metadata operations, and live event delivery. Access rules are applied at the session level: managers interact only with their assigned feeds, while administrators keep a broader governance role. This prevents the dashboard from showing controls that the current user should not access.

Live transport is selected according to the source type. IP camera feeds prioritize WebRTC because the monitoring screen benefits from lower latency. Local MP4 videos use MJPEG playback, which is simpler and sufficient for file-based demonstrations. When a preferred transport is unavailable, the backend reports the condition explicitly so the dashboard can show a fallback path instead of waiting silently.

Runtime control coordinates feed state, saved configuration, worker lifecycle, and live event broadcasting. The dashboard receives feed status changes, metric updates, alerts, and warnings through the live channel. For local MP4 videos, a shared MJPEG

Activity Diagram: Camera Discovery

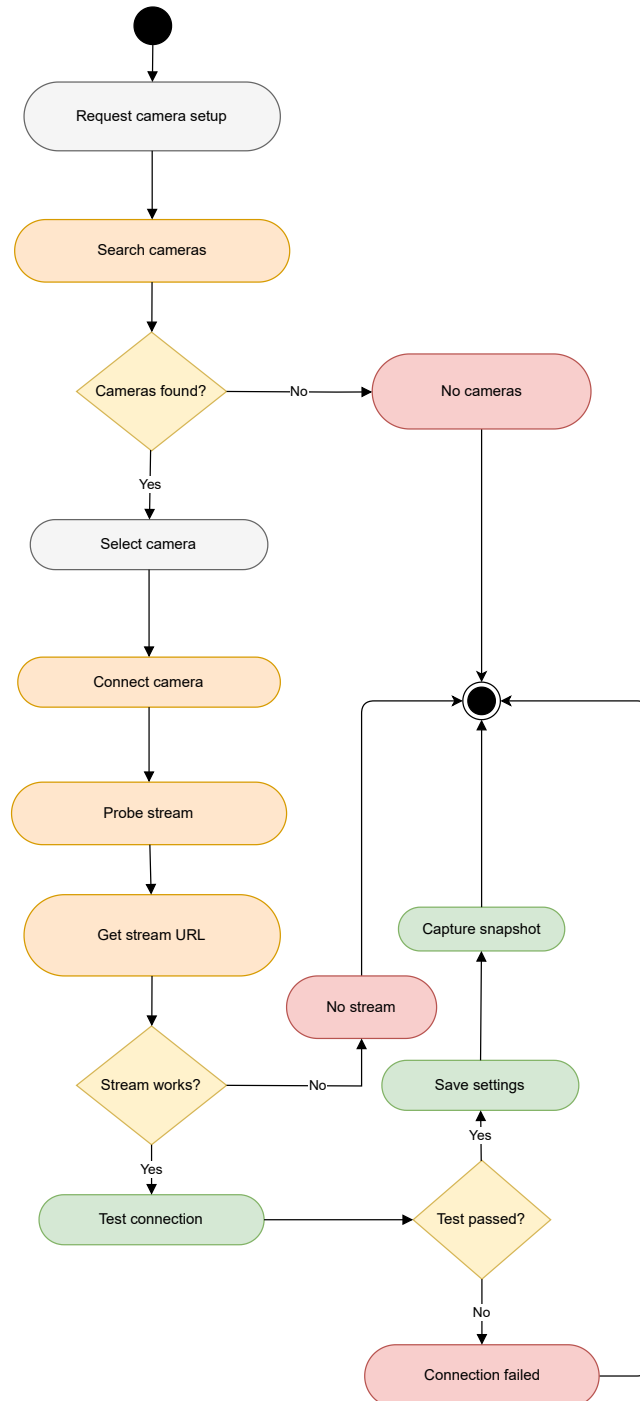


Figure 4.2: Camera discovery activity diagram: ONVIF discovery, RTSP resolution, and stream validation.

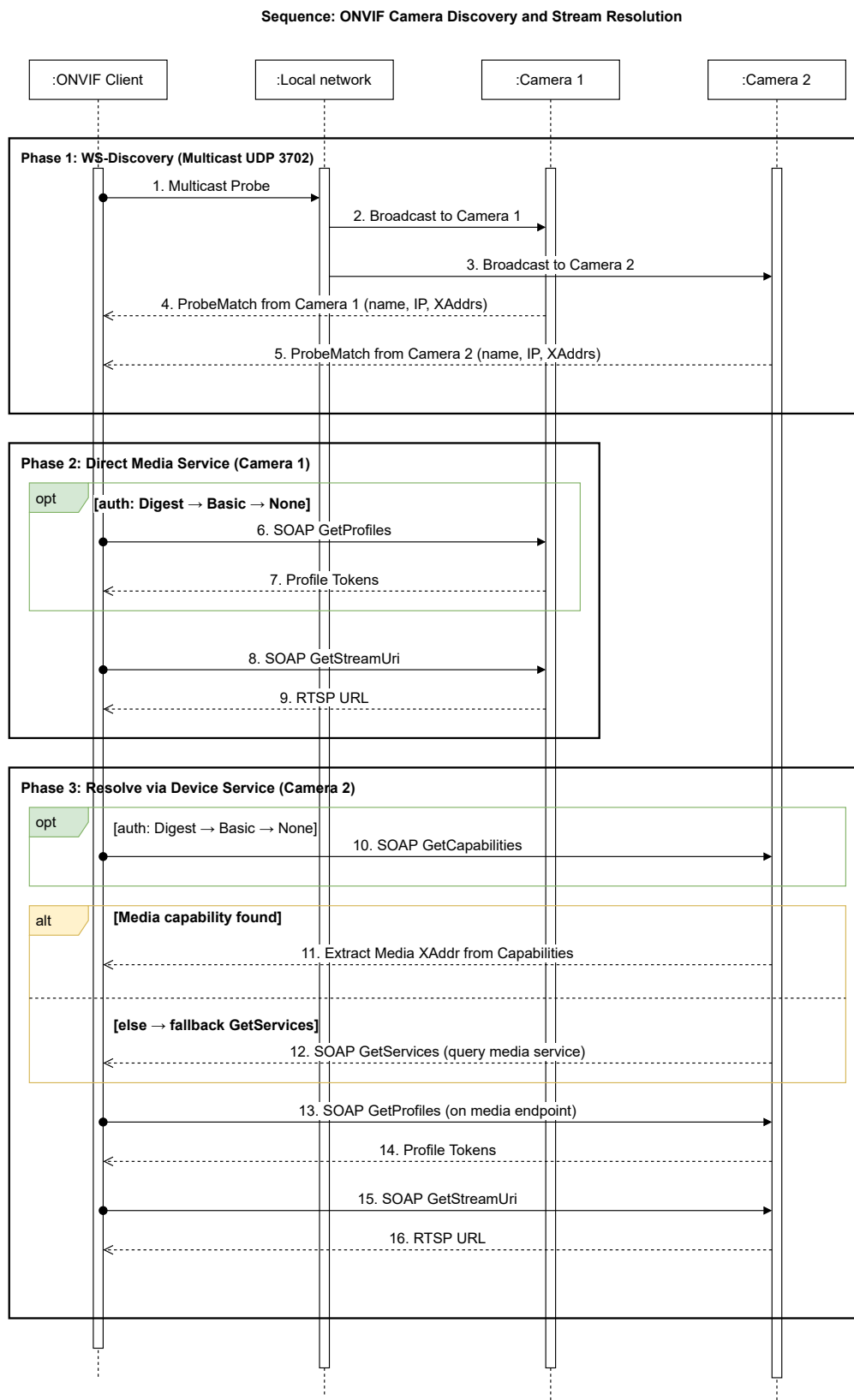


Figure 4.3: ONVIF protocol sequence: WS-Discovery broadcast, direct media service resolution, and device service indirection.

stream preserves viewing continuity during runtime mode transitions.

4.4.3 Persistence, State Recovery, and System Resilience

SQLite stores establishments, managers, feed configurations, video sessions, alert history, and system state. The structure keeps feed definitions attached to the correct establishment and manager, which is important when several feeds or users exist in the same deployment.

On startup, saved feed definitions are reloaded and unfinished sessions are reset to a consistent state. This avoids orphaned sessions and lets the application recover its last known configuration after a restart. In practice, Sprint 2 showed that camera connectivity is not only about reaching an RTSP address; the system also needs to remember feed state, report transport failures clearly, and recover from interruptions without losing the manager's configuration.

4.5 Conclusion

Sprint 2 turned the perception prototype into a managed service layer for camera discovery, feed lifecycle control, live events, and video delivery. The work addressed three project-level challenges: connecting heterogeneous sources through ONVIF and RTSP, keeping access and feed state consistent, and providing fallback-aware live delivery for the dashboard.

Table of Content

5.1	Introduction	46
5.2	Sprint Backlog	46
5.3	Design	48
5.3.1	Visual Design and Interface Architecture	48
5.3.2	Dashboard Data Flow and Integration Architecture	48
5.3.3	Design Rationale	50
5.4	Implementation	50
5.4.1	Project Setup and Layout Skeleton	50
5.4.2	Styling and Reusable Interface Rules	50
5.4.3	API Integration	50
5.4.4	Authentication and Role-Based Navigation	51
5.4.5	Dashboard State Orchestration and Live Update Reliability	53
5.4.6	Live Video Delivery and Transport Selection	53
5.4.7	Dashboard UI Integration	54
5.4.8	Manager Onboarding Workflow	54
5.4.9	Analytics and Historical Reporting	59
5.4.10	Integration Testing	60
5.5	Conclusion	61

5.1 Introduction

Sprint 3 turned the backend capabilities from Sprint 2 into a manager-facing web dashboard. The work began with the visual structure of the application, then moved to authentication, feed onboarding, live metrics, transport selection, and fallback behavior.

The main objective was to make monitoring usable in one place. A manager should be able to sign in, prepare a feed, define the queue zone, launch monitoring, follow live indicators, and review analytics without switching between separate tools.

5.2 Sprint Backlog

Sprint 3 combined interface design with live system integration. The design work defined the visual language, page structure, and reusable interaction patterns. The integration work connected those screens to authentication, feed management, Web-Socket updates, video transport, and fallback polling. Table 5.1 lists the manager and administrator needs together with the implementation work that supported them.

SPRINT 3: WEB DASHBOARD AND LIVE MONITORING

UC ID	User Story	Implementation Work
UC-3.1	As a manager, I want a coherent dashboard interface so that setup, monitoring, and analytics can be used from one place.	Define colors, typography, spacing, reusable component rules, and shared page scaffolding.
UC-3.2	As an administrator, I want to create and maintain manager accounts so that system access is centrally controlled.	Build the administrator dashboard for manager account creation, editing, activation, deactivation, and password reset.
UC-3.3	As a manager, I want protected sign-in and role-based navigation so that each user sees only the screens matching their responsibility.	Connect login state, protected routes, feed setup, zone selection, model selection, and feed control actions to the dashboard.
UC-3.4	As a developer, I want to implement WebSocket-driven live state updates and dashboard orchestration so that metric changes are immediately visible without page refresh.	Add WebSocket-driven live state updates, dashboard orchestration, and scenario-based integration tests.
UC-3.5	As a manager, I want live dashboard updates and fallback polling so that the interface remains informative during temporary interruptions.	Add fallback polling, stale-data guards, and error-state handling for live monitoring workflows.
UC-3.6	As a manager, I want a suitable video transport chosen automatically so that the feed remains usable when conditions change.	Add WebRTC negotiation, MJPEG fallback, transport selection, and reconnection logic.
UC-3.7	As a manager, I want historical analytics and exportable indicators so that repeated congestion patterns can be reviewed and shared.	Implement the analytics page, KPI summaries, charts, filters, and CSV export behavior.

Table 5.1: Sprint 3 backlog

5.3 Design

5.3.1 Visual Design and Interface Architecture

The interface follows a “Terminal Minimal” style suited to monitoring work. Dark surfaces reduce visual fatigue during long sessions, while semantic colors distinguish normal, warning, and critical states. Cards and panels are used to group feed summaries, KPI values, recent activity, and configuration steps without changing the manager’s mental model from page to page.

The application is organized around four main pages: dashboard, analytics, admin panel, and authentication. Defining this navigation early helped separate design decisions from live-data behavior. Once the main screens were stable, integration work could focus on whether the dashboard reacted correctly to feed changes, alerts, and transport failures.

5.3.2 Dashboard Data Flow and Integration Architecture

The dashboard data flow starts after authentication. The interface retrieves feed configuration and system health, then opens a persistent WebSocket connection for live updates. The backend sends an initial metrics snapshot, followed by feed lifecycle events, metric changes, alerts, and warnings. Feed actions such as launch, stop, or update are sent as REST commands, then reflected back through the live event stream.

This flow gives the dashboard two ways to remain informative. During normal operation, WebSocket events keep the metrics current without a page refresh. If live events stop arriving, the dashboard switches to periodic polling so the manager can still see feed state and recent metrics. For video playback, WebRTC is used for IP camera feeds when available, while MJPEG remains available for local videos and fallback views.

Figure 5.1 illustrates this complete data flow during setup, runtime operation, and transport negotiation.

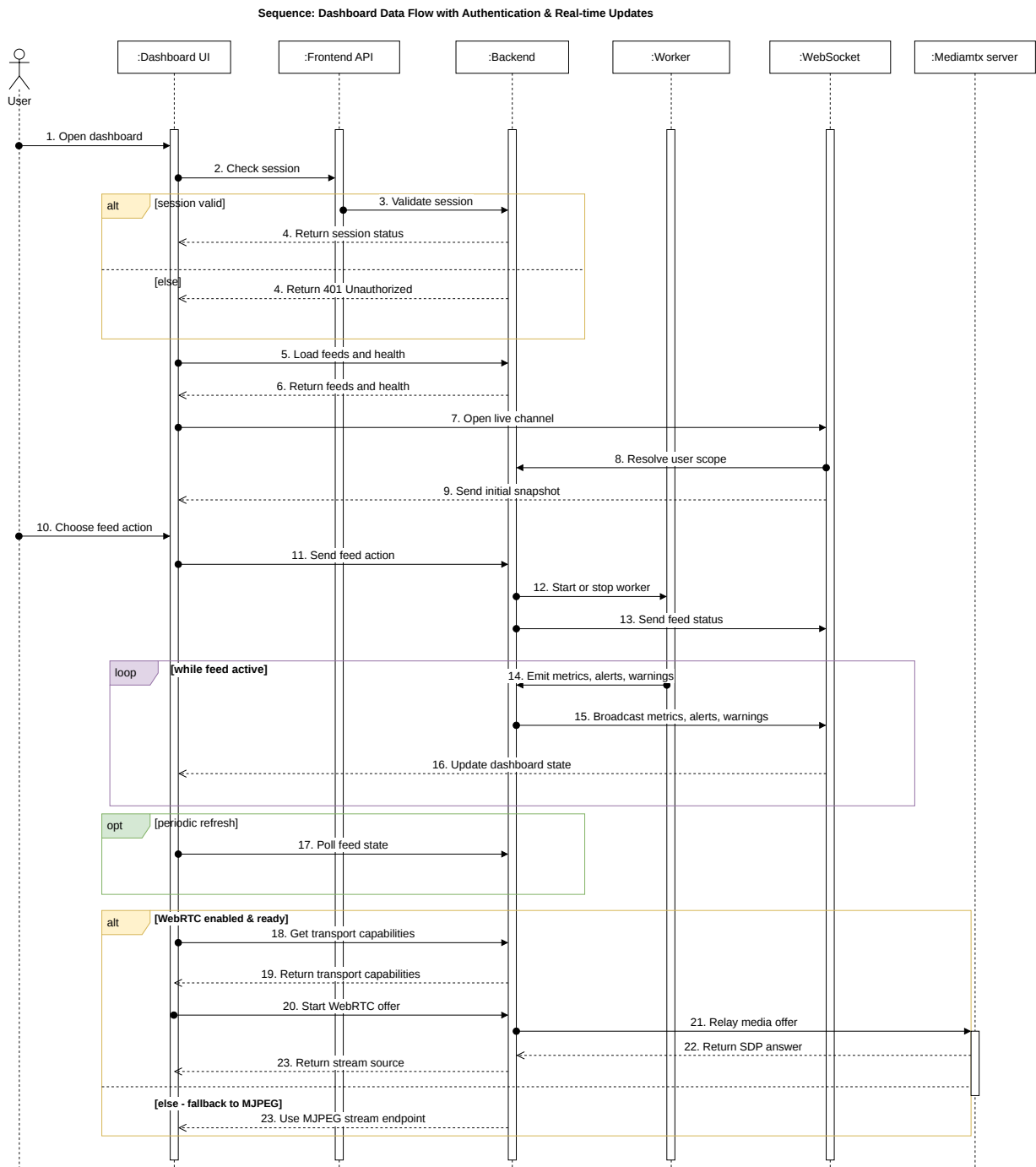


Figure 5.1: Dashboard data flow sequence: authentication, live metrics, fallback polling, and video transport.

5.3.3 Design Rationale

The design-first approach reduced confusion during integration. Because navigation, page responsibilities, and repeated interface elements were already defined, later work could focus on behavior: whether feeds launched correctly, whether metrics updated at the right time, and whether fallback states were understandable to the manager.

5.4 Implementation

5.4.1 Project Setup and Layout Skeleton

The design specification was translated into an application skeleton that includes protected navigation, the main dashboard layout, and shared interface elements across pages.

The page order follows the operational workflow: onboarding comes before configuration, configuration comes before launch, and launch leads into supervision. This makes the dashboard feel like one continuous monitoring path rather than a collection of unrelated screens.

5.4.2 Styling and Reusable Interface Rules

The visual system was defined once and reused across the dashboard. Spacing, typography, status colors, surfaces, and panel hierarchy follow the same rules in onboarding, monitoring, analytics, and administration screens. This helped avoid design drift as more live behavior was added.

5.4.3 API Integration

The dashboard communicates with the backend through a dedicated integration boundary. Feed lists, dashboard summaries, authentication state, and management actions are

handled as explicit responses rather than informal page-specific data. This keeps the user interface focused on presentation and interaction while the communication layer handles request state and errors.

5.4.4 Authentication and Role-Based Navigation

Sprint 3 integrated authentication and role-based access into the dashboard. Session changes clear cached dashboard data, protected routes redirect users according to role, administrators reach the manager administration interface, and managers reach the monitoring dashboard. The purpose is simple from the user's perspective: each role sees the screens that match its responsibilities.

Three key screens support this flow: the manager/admin sign-in screen, the manager account creation screen, and the administrator account-governance interface. These screens are shown in Figures 5.2, 5.3, and 5.4. Together, they keep authentication, account creation, and role-based routing within the same product experience.

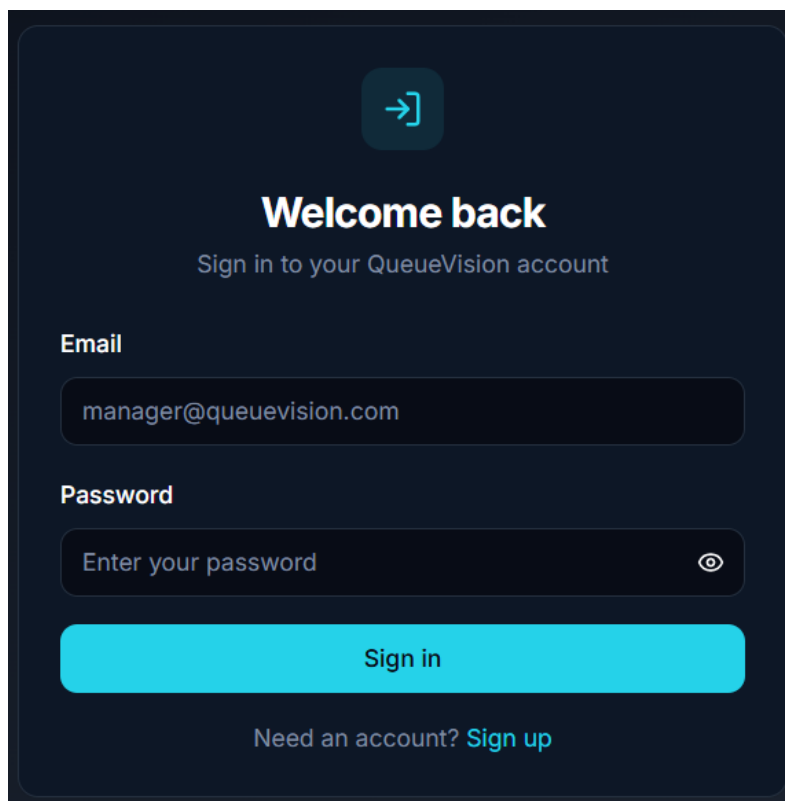
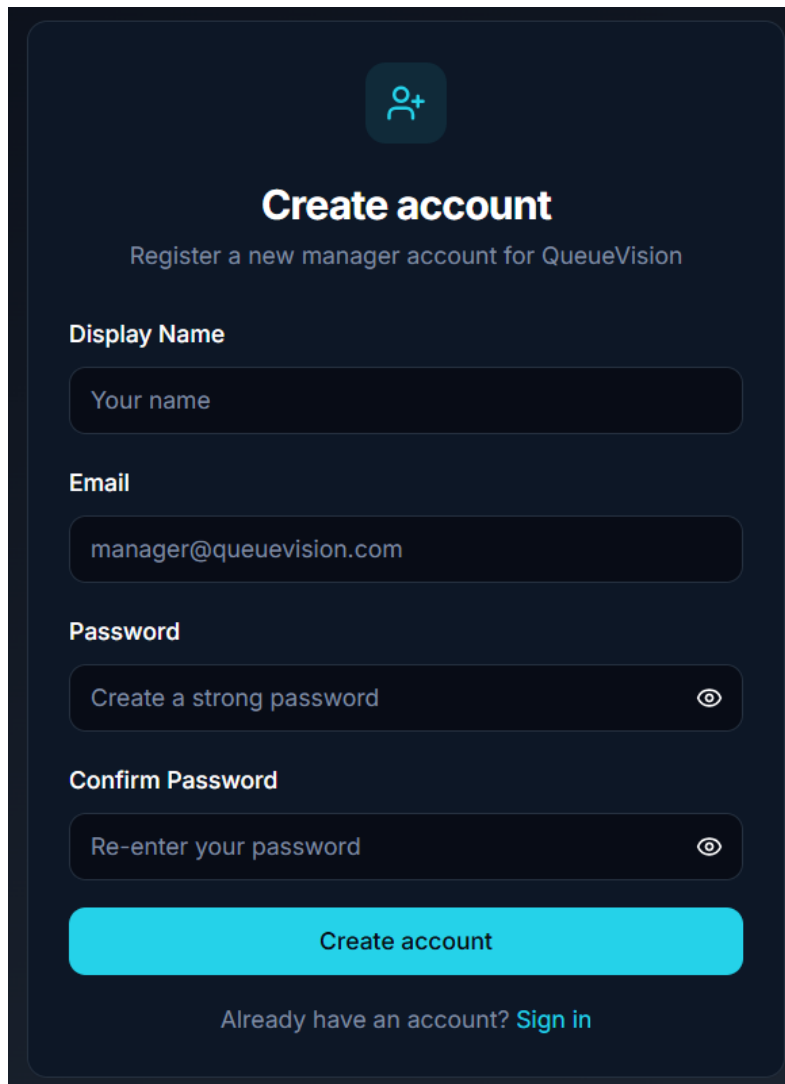


Figure 5.2: Manager/admin sign-in screen for dashboard access.



The image shows a 'Create account' form for QueueVision. At the top is a user icon. The title 'Create account' is centered, followed by the subtitle 'Register a new manager account for QueueVision'. The form contains four input fields: 'Display Name' with placeholder 'Your name', 'Email' with placeholder 'manager@queuevision.com', 'Password' with placeholder 'Create a strong password' and an eye icon, and 'Confirm Password' with placeholder 'Re-enter your password' and an eye icon. A large blue 'Create account' button is at the bottom, with a link 'Already have an account? Sign in' below it.

Create account

Register a new manager account for QueueVision

Display Name

Your name

Email

manager@queuevision.com

Password

Create a strong password

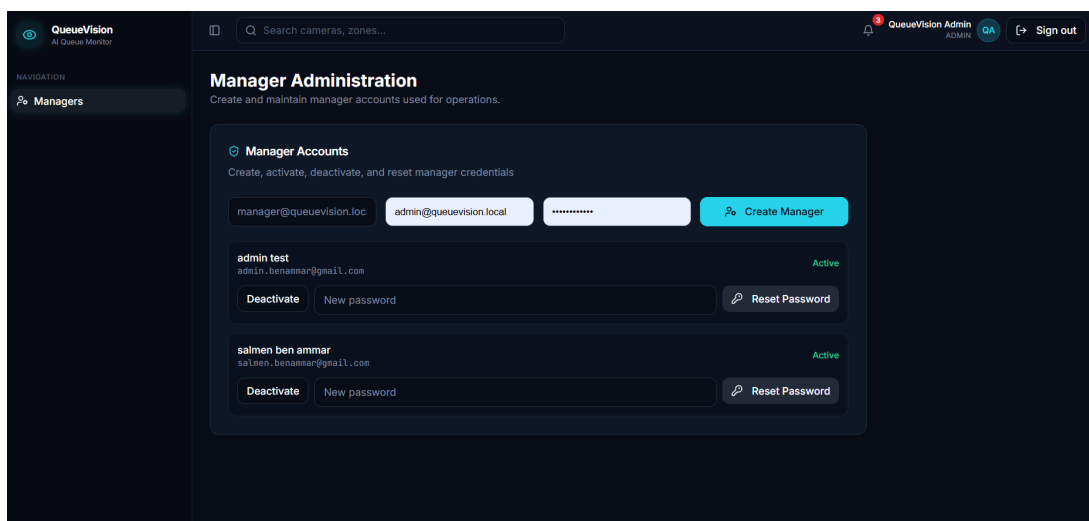
Confirm Password

Re-enter your password

Create account

Already have an account? [Sign in](#)

Figure 5.3: Manager account creation screen.



The image shows the 'Manager Administration' interface. The top header includes the QueueVision logo, a search bar, and user information 'QueueVision Admin QA' with a 'Sign out' button. The left sidebar has a 'Managers' link. The main content area is titled 'Manager Administration' with a subtitle 'Create and maintain manager accounts used for operations.' Below this is a 'Manager Accounts' section with a description 'Create, activate, deactivate, and reset manager credentials'. It features a form to 'Create Manager' with fields for email, username, and password. Below the form are two existing accounts: 'admin test' and 'salmen ben ammar', each with 'Deactivate' and 'Reset Password' buttons. The status 'Active' is shown next to each account name.

QueueVision
AI Queue Monitor

Search cameras, zones...

QueueVision Admin QA Sign out

Managers

Manager Administration
Create and maintain manager accounts used for operations.

Manager Accounts
Create, activate, deactivate, and reset manager credentials

manager@queuevision.loc admin@queuevision.local Create Manager

admin test
admin.benamar@gmail.com Active
Deactivate New password Reset Password

salmen ben ammar
salmen.benamar@gmail.com Active
Deactivate New password Reset Password

Figure 5.4: Administrator manager-administration interface.

5.4.5 Dashboard State Orchestration and Live Update Reliability

The dashboard combines feed data into operational indicators such as online feed count, total people, average wait time, and alert count. These indicators are recalculated as feed events arrive, so the monitoring screen reflects the current state of the system rather than a static page load.

Live reliability relies on several behaviors:

- reconnection delays to avoid repeated rapid retries during outages,
- heartbeat monitoring to detect connection loss,
- stale-data guards so old metrics are not presented as fresh,
- fallback polling when live events become unavailable,
- batched updates to keep the interface responsive.

When the WebSocket connection drops, the dashboard switches to polling and keeps the monitoring surface usable. In this context, continuous visibility is more important than using a single ideal transport. The manager still needs enough information to understand feed state and queue pressure during a temporary network interruption.

5.4.6 Live Video Delivery and Transport Selection

Video transport is selected according to the source type and availability. IP cameras use WebRTC when possible because it gives lower latency for live monitoring. Local MP4 videos use MJPEG, which is simpler and more appropriate for file-based playback.

For IP camera feeds, the browser negotiates a receiving stream with the backend and then transitions to live playback. If the primary transport fails repeatedly, the dashboard stops retrying aggressively and moves to a fallback view. This behavior prevents the interface from depending on one transport mechanism and keeps the feed area meaningful even when WebRTC is unavailable.

5.4.7 Dashboard UI Integration

The dashboard integrates live feed controls, setup dialogs, onboarding, KPI visualizations, and charts. Onboarding supports both uploaded videos and ONVIF sources. For cameras, the manager discovers devices, supplies per-camera credentials, resolves RTSP streams, validates the selection, captures a snapshot for zone definition, and stages the feed for launch. Figure 5.5 shows the operational monitoring surface.

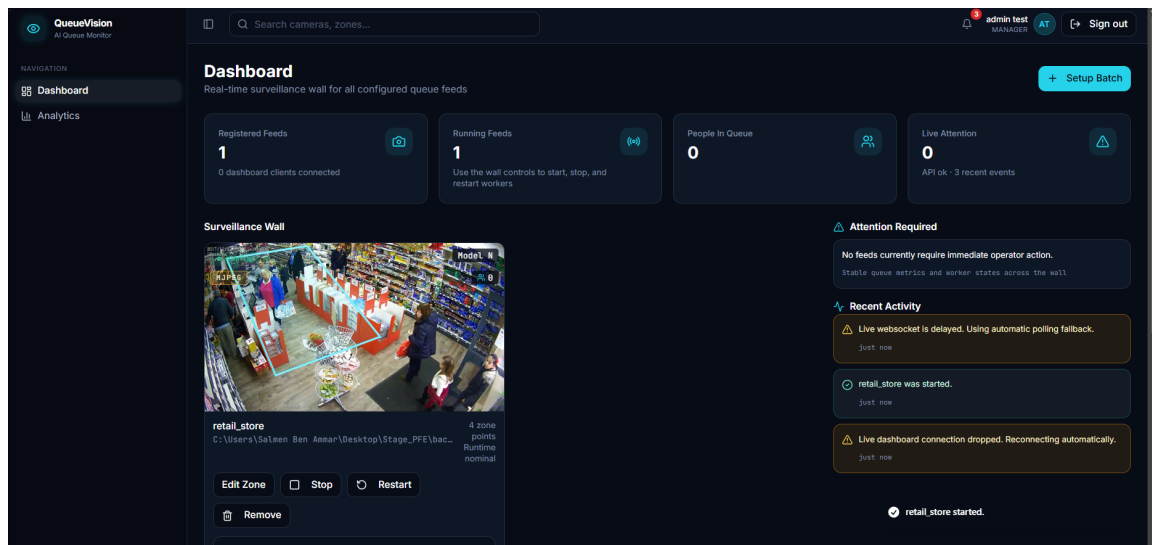


Figure 5.5: Manager dashboard: live surface, feed summaries, and recent activity.

5.4.8 Manager Onboarding Workflow

The manager onboarding flow covers the full path from feed setup to feed confirmation. Figure 5.6 shows the main phases: source selection, source configuration, connection testing, zone definition, model selection, and final review.

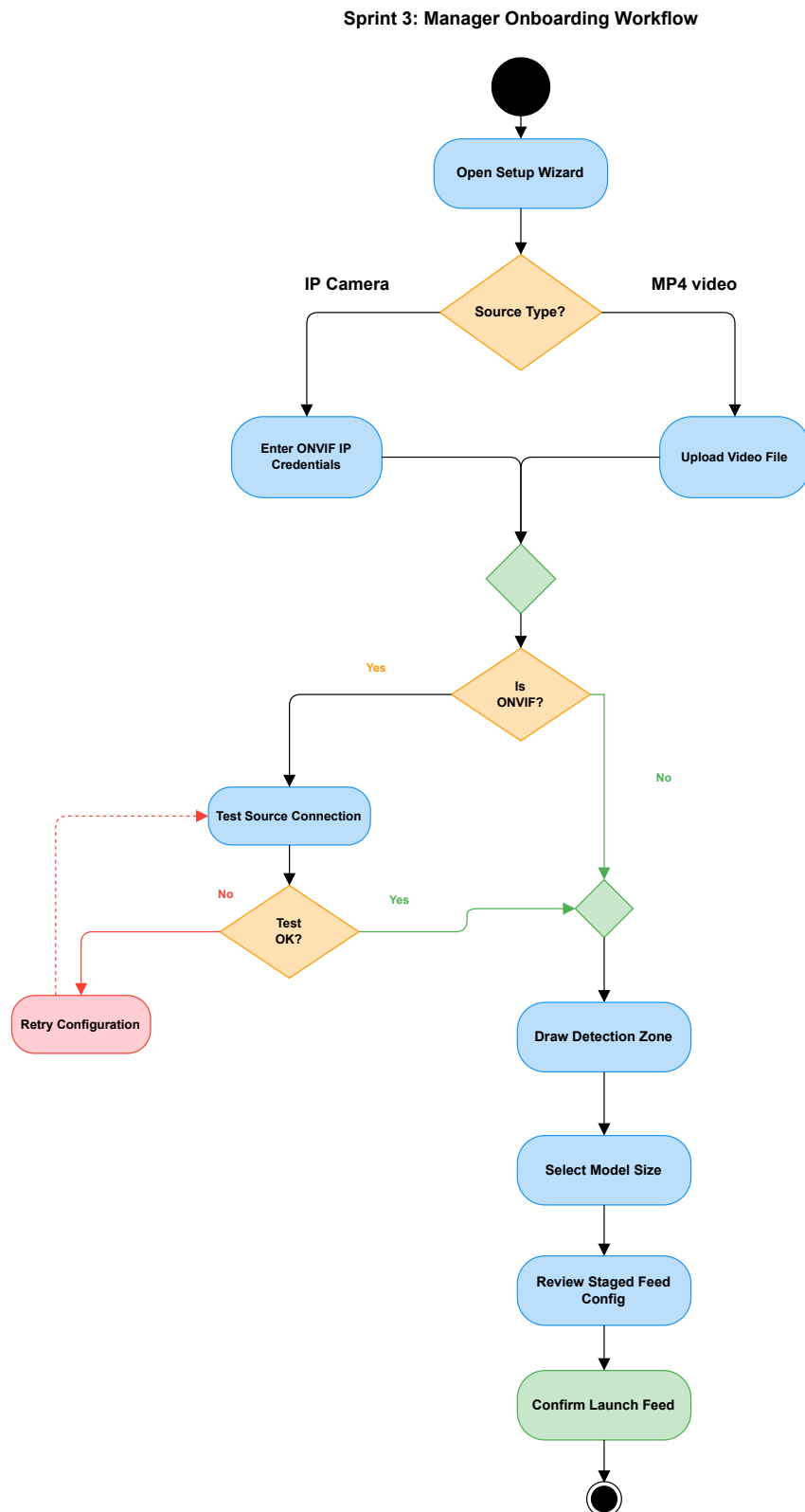


Figure 5.6: Manager onboarding activity diagram: feed setup, zone definition, model selection, and launch confirmation.

The workflow execution involves several screens. First, the feed setup screen prepares an uploaded video or ONVIF source:

Stage Sources

Enter the feed information for one source, optionally attach metadata, then continue through zone tracing, model selection, and review.

One source is prepared at a time. Finish this source, then continue to zone tracing and model selection.

Feed name

Checkout 1

Source type

Local MP4 ONVIF Discovery

Upload a local video
Supported formats: MP4, MOV, AVI, and MKV. The backend stores the file and uses its path automatically.

Choose Video File No video selected yet

Local videos are configured one at a time so each feed keeps a single zone and model.

Step 1 uploads the source file into the backend workflow. Step 2 lets you draw the queue polygon on the video frame. Step 3 selects the model size.

Pick one local video to see it listed here.

Cancel Continue to Zone

Figure 5.7: Feed setup screen used to prepare an uploaded video or ONVIF camera source before zone tracing and model selection.

Then the zone is defined through the tracing interface:

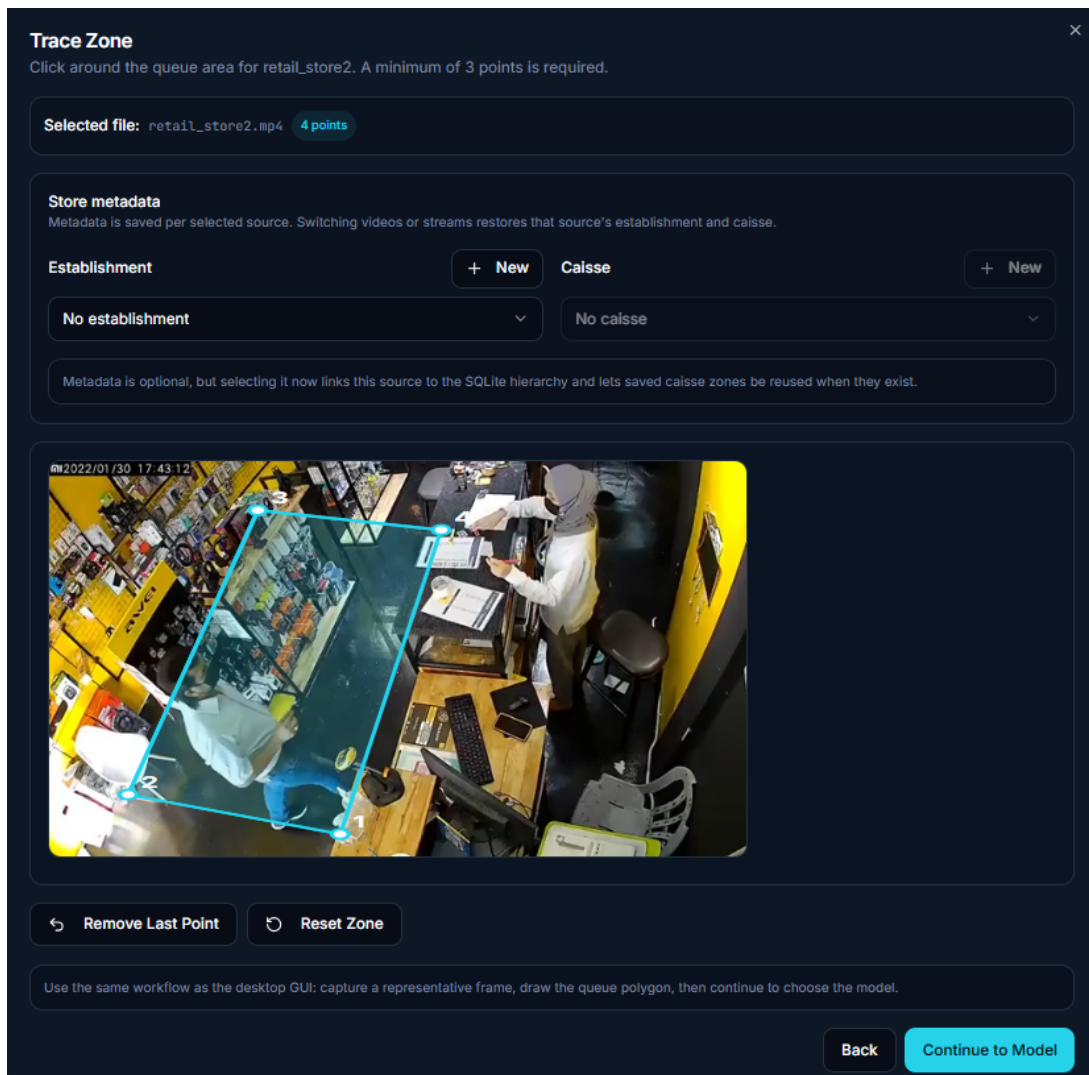


Figure 5.8: Zone tracing screen used to define the queue polygon for the selected feed.

Model selection precedes the final confirmation. The manager chooses the YOLO size here:

Choose Model

Finalize retail_store2 with the YOLO model size you want to run.

SOURCE TYPE	FEED NAME	ZONE POINTS	ESTABLISHMENT
Uploaded video	retail_store2	4	Not assigned
CAISSE			Not assigned

N - Nano
Best for rapid iteration
Fastest setup for quick validation and lightweight machines.

S - Small
Good default for development
Balanced choice when you want better detection quality without a large hit.

M - Medium
Solid for denser queues
More accurate across crowded scenes with a moderate compute budget.

L - Large
Heavier runtime cost
Higher accuracy for harder scenes when your GPU budget allows it.

X - X-Large
Slowest option
Maximum accuracy target for controlled, powerful environments.

[Back](#) [Continue to Review](#)

Figure 5.9: Model selection screen used to choose the YOLO size before launching the feed.

Finally, the staged feed or batch is confirmed before launch:

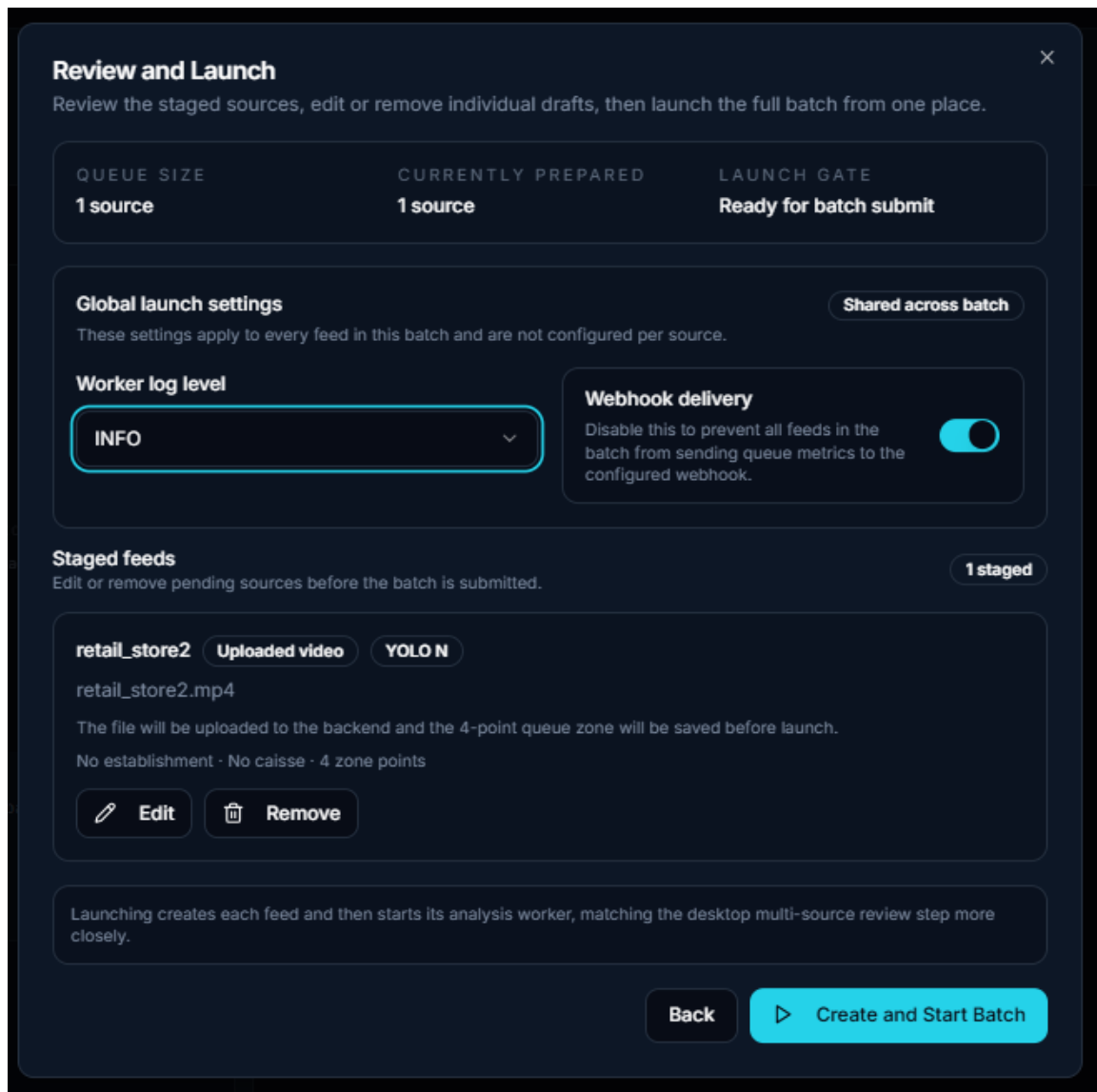


Figure 5.10: Review and launch panel used to confirm staged feeds before batch creation or launch.

The practical result is that a manager can move from onboarding to monitoring without leaving the dashboard context. Setup, validation, zone definition, launch, and supervision remain part of the same workflow.

5.4.9 Analytics and Historical Reporting

The analytics view complements live supervision with historical reporting. It aggregates archived alerts and queue metrics so managers can identify repeated congestion patterns and compare queue zones over time.

Four KPI cards summarize average wait time, peak queue length, stability score, and total alerts. A period selector controls hourly, daily, or weekly granularity, while date filters isolate a specific window. The dashboard visualizes wait and queue trends, ranks top and bottom zones, and breaks down alerts by severity. CSV export supports external reporting. During early pilots when the archive is empty, the interface uses active-feed data so the analytics page remains useful for demonstration and review.

Figure 5.11 shows the analytics dashboard.

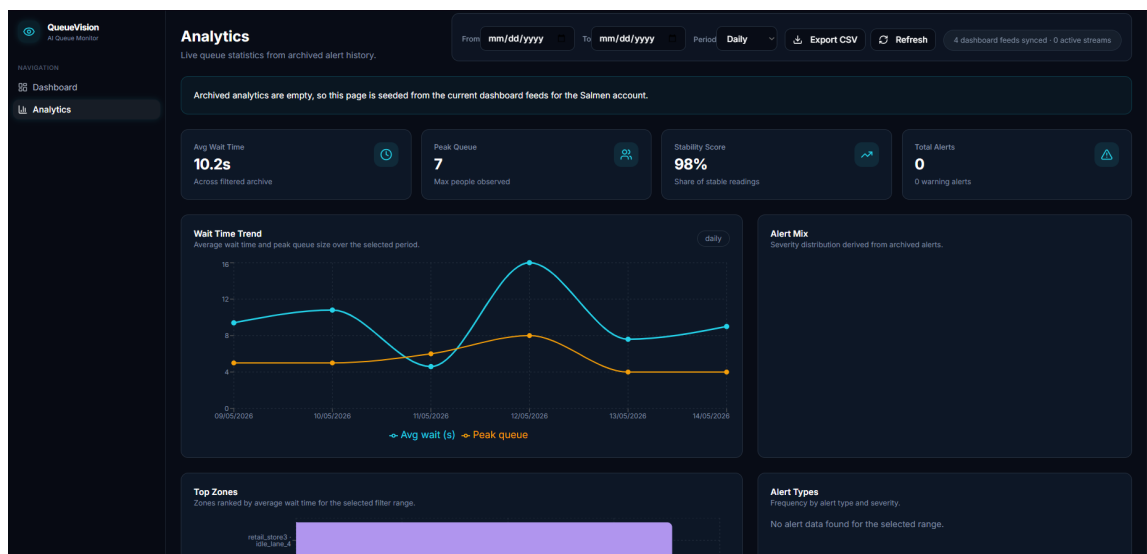


Figure 5.11: Analytics dashboard: KPI summaries, trends, zone rankings, and export.

5.4.10 Integration Testing

The dashboard was validated through scenarios covering setup flows, live event handling, transport fallback, feed routing, and error handling.

These tests were useful because the dashboard is not correct just because it renders. It also has to behave properly when live events reconnect, a feed changes state, a route is protected, or a fallback transport is activated. The validation therefore focused on the workflows a manager would actually experience during monitoring.

5.5 Conclusion

Sprint 3 delivered the manager-facing monitoring interface and connected it to the live system behavior prepared in Sprint 2. The dashboard now supports authentication, feed setup, ONVIF onboarding, zone selection, model choice, launch review, live metrics, video transport selection, analytics, and administration screens.

Sprint 4: Automation, Evaluation, and Optimization

Table of Content

6.1	Introduction	63
6.2	Sprint Backlog	63
6.3	Design	64
6.4	Implementation	65
6.4.1	Alert Payload and Delivery Architecture	65
6.4.2	Telegram Integration and Notification Delivery	66
6.4.3	Asynchronous Alert Dispatch and Deduplication	67
6.4.4	n8n Workflow Structure and Validation	67
6.5	Tests and Evaluation	67
6.5.1	Evaluation Strategy and Scope	67
6.5.2	Integrated System Verification	69
6.5.3	Runtime Optimization and Performance Tuning	69
6.6	Conclusion	70

6.1 Introduction

Sprint 4 completed the monitoring loop by connecting queue alerts to an external notification channel and validating the system as a whole. Until this sprint, the project could detect queues, estimate waiting time, expose live services, and display the dashboard. The remaining step was to make alerts reach operational staff and to verify that the full chain behaved correctly under realistic conditions.

The sprint therefore combined two types of work: alert automation through n8n and Telegram, and final evaluation of perception, live monitoring, transport fallback, and notification delivery.

6.2 Sprint Backlog

Sprint 4 focused on alert automation and system-wide verification. Alert automation required a validated payload, secure workflow access, cooldown handling, and Telegram delivery. Verification required testing the integrated system, checking streaming stability, tuning runtime responsiveness, and improving recovery from temporary failures. Table 6.1 lists the user stories and implementation work that drove this sprint.

UC ID	User Story	Implementation Work
UC-4.1	As a developer, I want to implement webhook dispatch with validated alert payloads and proper authorization so that alerts securely reach the notification workflow.	Define the webhook payload schema, validate it before dispatch, and protect delivery with shared-secret authorization.
UC-4.2	As a manager, I want Telegram notifications routed to the on-site operator so that field action can be triggered through a familiar channel.	Create and configure the Telegram bot, store its credential in n8n, and configure the workflow branches.
UC-4.3	As a developer, I want to implement resilient webhook delivery with retries, cooldown deduplication, and audit logging so that alert delivery is reliable and traceable.	Add retry logic, timeout handling, cooldown checks, and delivery logging for webhook notifications.
UC-4.4	As a developer, I want alert routing validated under realistic scenarios so that notification behavior is trustworthy before deployment.	Prepare PowerShell validation scripts for single-alert, multi-feed, and execution polling scenarios.
UC-4.5	As a developer, I want the complete monitoring chain tested and runtime behavior tuned so that the platform is reliable under realistic conditions.	Run backend and frontend integration checks for lifecycle actions, streaming, API contracts, and fallback behavior; apply configurable image size, frame stride, async dispatch, playback pacing, worker supervision, frame handoff continuity, and WebSocket reconnection improvements.

Table 6.1: Sprint 4 backlog

6.3 Design

The alert workflow was designed to avoid two opposite problems: missing important queue alerts and overwhelming staff with repeated notifications for the same condition. When an alert arrives, the workflow first checks that the message is authorized. It then applies cooldown logic, archives the alert in the database, and sends the validated

notification to Telegram.

This separation makes the workflow easier to reason about. Authorization protects the entry point, cooldown controls repetition, archival supports traceability, and Telegram delivery reaches the on-site staff channel. Figure 6.1 illustrates the complete workflow logic.

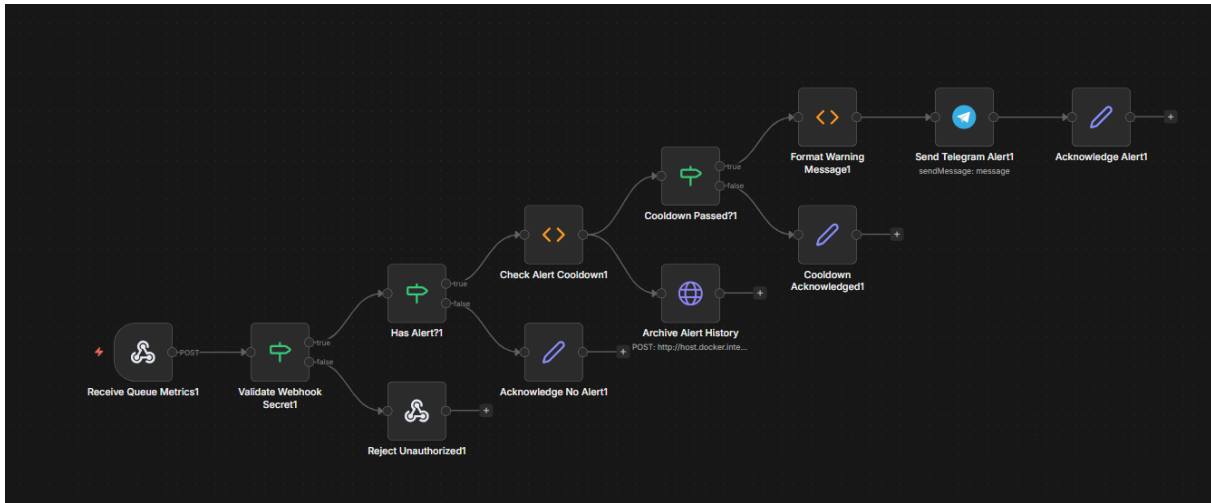


Figure 6.1: Sprint 4 n8n workflow for routing queue alerts through cooldown checks, archival, and Telegram delivery.

6.4 Implementation

6.4.1 Alert Payload and Delivery Architecture

The alert payload contains the information needed by the automation workflow and the Telegram message: timestamp and frame context, queue metrics (λ , μ , and expected wait time), alert flags, and severity. Payload validation happens before dispatch so incomplete or malformed messages are not sent to external workflow tools.

Delivery reliability was handled through retry behavior, timeouts, shared-secret validation, and dispatch logging. Successful and failed delivery attempts are kept for traceability, which is important when a manager needs to know whether a threshold violation was actually sent to the notification workflow.

6.4.2 Telegram Integration and Notification Delivery

A Telegram bot was configured as the notification channel, with its credential stored in n8n rather than directly inside the exported workflow. This keeps the workflow portable and avoids exposing secrets in shared project artifacts. Alerts that pass authorization and cooldown checks are formatted for human readability and sent to the operational group chat. An example of delivered alert formatting is shown in Figure 6.2.

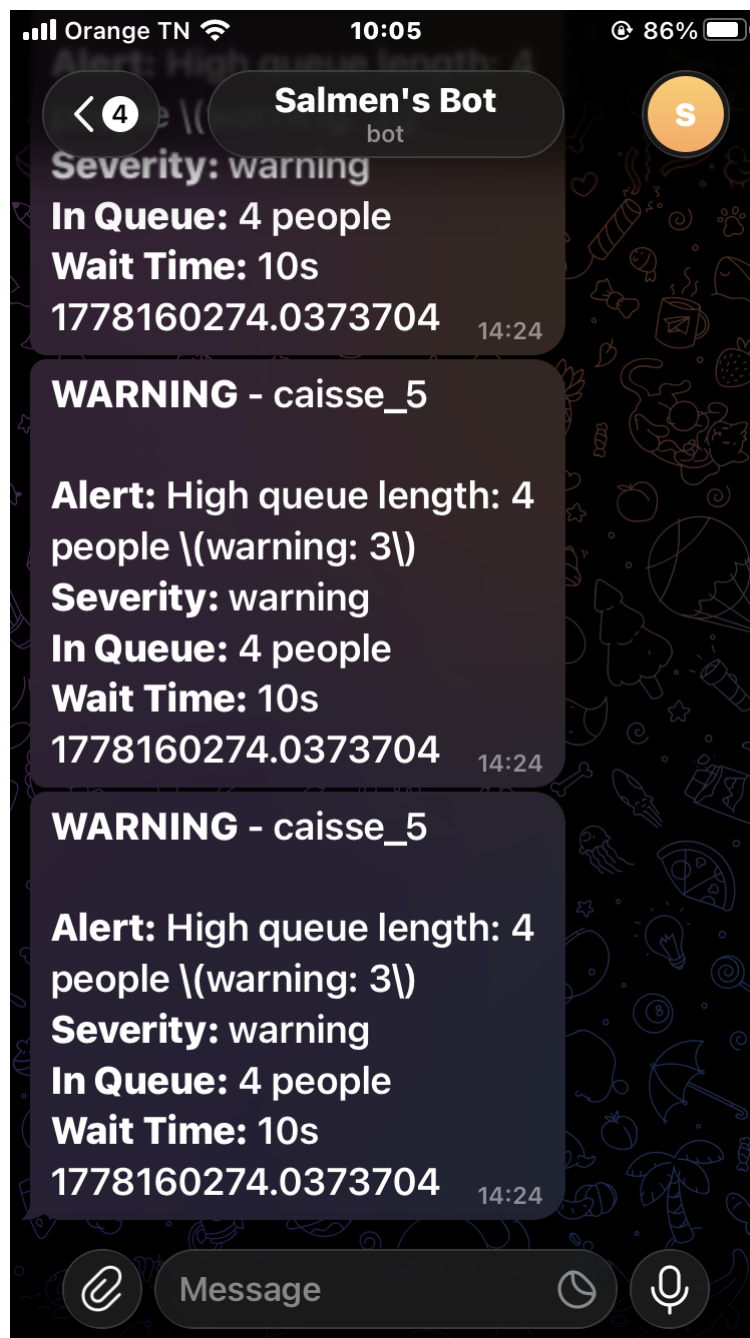


Figure 6.2: Example Telegram alert delivered.

6.4.3 Asynchronous Alert Dispatch and Deduplication

Alert delivery was separated from the perception loop so that network latency would not slow frame processing. This matters because webhook calls depend on external services and can be delayed or fail temporarily. Queue analysis should continue even if the notification path is slower than expected.

Deduplication reduces repeated notifications when the same threshold violation remains active across several metric windows. A new alert is still sent when the condition changes or a new threshold violation appears, but persistent conditions do not flood the Telegram group.

6.4.4 n8n Workflow Structure and Validation

The n8n workflow separates three alert states: no alert, suppressed alert during cooldown, and delivery-ready alert. This structure made validation clearer because each branch could be exercised independently.

PowerShell validation scripts were used for single-alert scenarios, multi-feed scenarios, and execution polling. These checks confirmed that authorization, cooldown routing, and Telegram delivery behaved as expected across different alert paths.

6.5 Tests and Evaluation

6.5.1 Evaluation Strategy and Scope

The evaluation strategy covered the main parts of the finished system: queue analytics, API behavior, dashboard responsiveness, transport fallback, and n8n routing. The most important observations were whether the perception pipeline stayed responsive, whether streams recovered after feed changes, whether alerts reached Telegram, and whether the dashboard stayed usable when WebSocket delivery was interrupted.

The tests combined focused checks for individual behaviors with integrated scenarios that followed the system from queue-state change to dashboard update or Telegram notification. Normal operation, temporary failures, feed lifecycle changes, and alert cooldown behavior were all included because these are the situations most likely to affect a real monitoring deployment.

Figure 6.3 illustrates the iterative evaluation and tuning cycle that characterized the sprint.

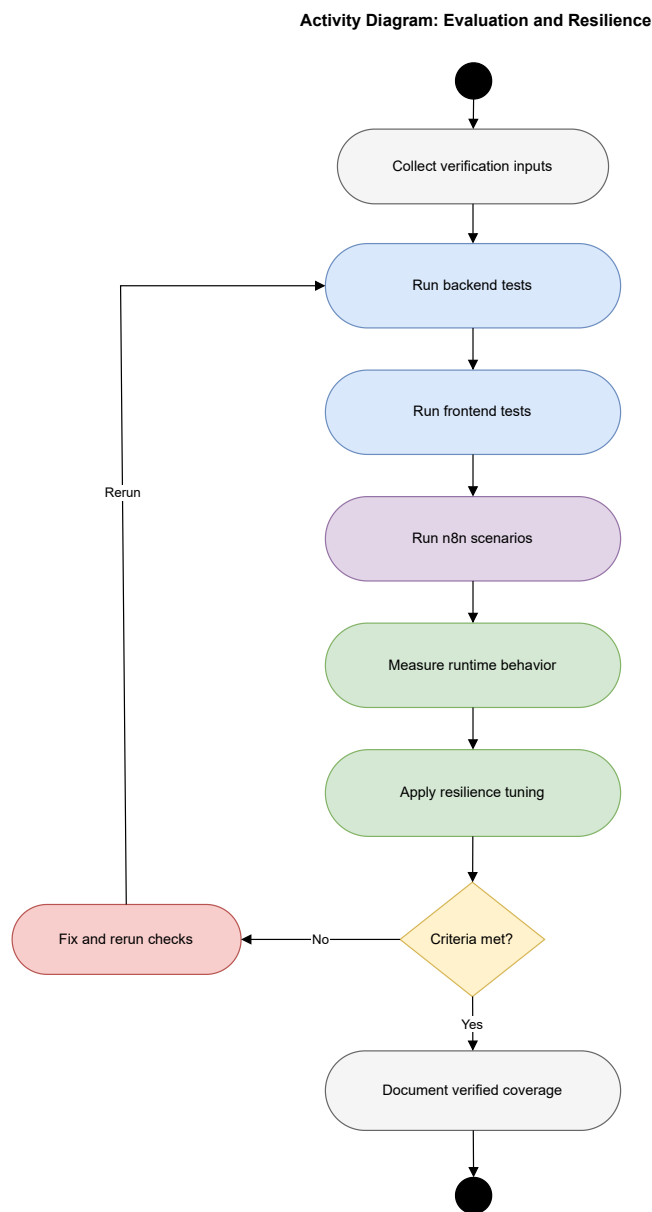


Figure 6.3: Sprint 4 evaluation and resilience activity diagram for tests, tuning, and recovery verification.

6.5.2 Integrated System Verification

Integration verification covered the complete monitoring chain: feed lifecycle actions, streaming behavior, live dashboard updates, webhook delivery, and Telegram notification routing. Error scenarios included authentication failures, WebSocket disconnections, WebRTC unavailability, and feed worker interruption.

The goal was not only to confirm that each feature worked in isolation, but also to check that failures in one part of the system did not make the whole platform unusable. For example, a WebSocket interruption should move the dashboard to polling, and a temporary notification failure should be recorded rather than silently ignored.

6.5.3 Runtime Optimization and Performance Tuning

Several runtime tuning options were added to improve responsiveness. Two key parameters control the inference behavior:

- **Image size (imgsz):** Resolution of frames sent to YOLO. Larger images (640×640) improve accuracy but cost more GPU; smaller images (320×320) are faster but miss details.
- **Frame stride:** How many frames to skip between detections. Stride=1 processes every frame; stride=2 processes every second frame; stride=3 is fastest but may miss events.

Table 6.2 summarizes the configurations evaluated during tuning and the measured impact on inference time, GPU load, and effective throughput.

imgsz	Stride	Inference Time	GPU Load	Effective FPS	Use Case
320	3	5 ms	18%	28	Fast processing, minimal VRAM usage
512	2	7 ms	24%	15	Balanced accuracy / speed (<i>default</i>)
640	1	11 ms	33%	13	High-accuracy, all frames processed

Table 6.2: Runtime configuration variants and measured performance characteristics.

6.6 Conclusion

Sprint 4 delivered the final monitoring-to-action loop. Queue alerts could be validated, routed through n8n, filtered by cooldown logic, and delivered to Telegram. The evaluation work confirmed the main integrated behaviors: live metrics, dashboard resilience, stream fallback, alert dispatch, and workflow routing.

General Conclusion

Table of Content

7.1	Summary	72
7.2	Objective Review	73
7.3	Future Improvements	73

7.1 Summary

This end-of-studies project produced a queue wait-time estimation platform that connects computer vision, queue analytics, web supervision, and alert automation. The project started from a practical problem: managers need more than a visual impression of a queue. They need indicators that show how the queue is evolving and when action is required.

In practical terms, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- adaptive wait-time outputs that respond to queue conditions,
- a secure authentication layer for manager sign-up, sign-in, session refresh, and administrator account governance,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live websocket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

Taken together, these elements form a monitoring-to-action loop: the system observes a queue, estimates its state, displays it to the manager, and sends alerts when the situation requires attention.

Across the four sprint chapters, the work progressed from perception and queue analytics to live services, then to the web dashboard, and finally to automation and validation.

7.2 Objective Review

The initial objectives were achieved at the project scope level:

1. **Real-time detection and tracking:** achieved through YOLO-based detection and ByteTrack identity continuity.
2. **Queue-zone counting:** achieved with polygon-zone management and in-zone identity filtering.
3. **Queue metric estimation:** achieved for arrival rate, service rate, and wait time.
4. **Adaptive wait-time estimation:** achieved through automatic formula switching based on queue load.
5. **Web monitoring:** achieved through a multi-feed dashboard with live updates, setup workflows, and fallback behavior.
6. **Automation integration:** achieved through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but the depth of long-duration evaluation under broader operational scenarios.

7.3 Future Improvements

Several extensions would strengthen the platform before wider deployment:

- **Mobile application:** developing a companion mobile application allowing operators and managers to monitor queue activity, consult statistics, and receive alerts remotely in real time.
- **In-platform messaging:** introducing a notification system enabling managers to communicate directly with operators, such as assigning available staff to queues

based on live crowd conditions.

- **Tracking accuracy improvement:** refining the person re-identification pipeline to reduce identity switches and tracking losses, particularly in dense or partially occluded queue scenarios.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation – yolo real-time object detection*, <https://docs.ultralytics.com/>, Accessed: 2026-03-01, 2026.
- [3] Waitwhile, *Waitwhile – virtual queue and appointment management*, <https://www.waitwhile.com/>, Accessed: 2026-02-24, 2026.
- [4] Qminder, *Qminder – queue management system*, <https://www.qminder.com/>, Accessed: 2026-02-24, 2026.
- [5] RetailNext, *Retailnext – in-store analytics and traffic intelligence*, <https://retailnext.net/>, Accessed: 2026-02-25, 2026.
- [6] Xovis, *Xovis – 3d sensor-based people flow and queue measurement*, <https://www.xovis.com/>, Accessed: 2026-02-25, 2026.
- [7] Roboflow, *Roboflow – computer vision platform*, <https://roboflow.com/>, Accessed: 2026-02-26, 2026.
- [8] Microsoft, *Visual studio code documentation*, <https://code.visualstudio.com/docs>, Accessed: 2026-02-21, 2026.
- [9] Git, *Git documentation*, <https://git-scm.com/doc>, Accessed: 2026-02-20, 2026.
- [10] GitHub, *Github documentation*, <https://docs.github.com/>, Accessed: 2026-02-20, 2026.
- [11] diagrams.net, *Diagrams.net (draw.io) documentation*, <https://www.diagrams.net/>, Accessed: 2026-02-23, 2026.

REFERENCES

- [12] Docker, *Docker documentation – containers and compose*, <https://docs.docker.com/>, Accessed: 2026-02-19, 2026.
- [13] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [14] OpenCV, *Opencv documentation*, <https://docs.opencv.org/>, Accessed: 2026-02-27, 2026.
- [15] S. Ramírez, *Fastapi documentation*, <https://fastapi.tiangolo.com/>, Accessed: 2026-03-17, 2026.
- [16] Pydantic, *Pydantic documentation*, <https://docs.pydantic.dev/>, Accessed: 2026-03-18, 2026.
- [17] React, *React documentation*, <https://react.dev/>, Accessed: 2026-04-14, 2026.
- [18] Microsoft, *Typescript documentation*, <https://www.typescriptlang.org/docs/>, Accessed: 2026-04-15, 2026.
- [19] Vite, *Vite documentation*, <https://vite.dev/guide/>, Accessed: 2026-04-16, 2026.
- [20] T. CSS, *Tailwind css documentation*, <https://tailwindcss.com/docs/>, Accessed: 2026-04-18, 2026.
- [21] W3C Web Real-Time Communications Working Group, *WebRTC 1.0: Real-time communication between browsers*, <https://www.w3.org/TR/webrtc/>, Accessed: 2026-03-27, 2025.
- [22] MediaMTX, *Mediamtx – media server documentation*, <https://github.com/bluenviron/mediamtx>, Accessed: 2026-03-24, 2026.
- [23] n8n, *N8n documentation – workflow automation*, <https://docs.n8n.io/>, Accessed: 2026-05-13, 2026.
- [24] Telegram, *Telegram bot api*, <https://core.telegram.org/bots/api>, Accessed: 2026-05-15, 2026.

REFERENCES

- [25] SQLite, *Sqlite documentation*, <https://www.sqlite.org/docs.html>, Accessed: 2026-03-20, 2026.
- [26] NumPy, *Numpy documentation*, <https://numpy.org/doc/>, Accessed: 2026-02-25, 2026.
- [27] pandas, *Pandas documentation*, <https://pandas.pydata.org/docs/>, Accessed: 2026-04-01, 2026.
- [28] SciPy, *Scipy documentation*, <https://docs.scipy.org/doc/scipy/>, Accessed: 2026-04-03, 2026.